Week 3

Operating System

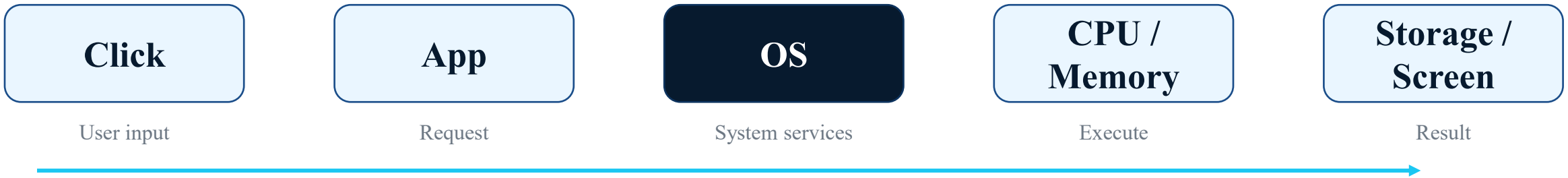
The invisible manager behind every digital device



DSI102 Introduction to Digital Technology
Dr. Nantapong Keandoungchun

What happens when you click an application?

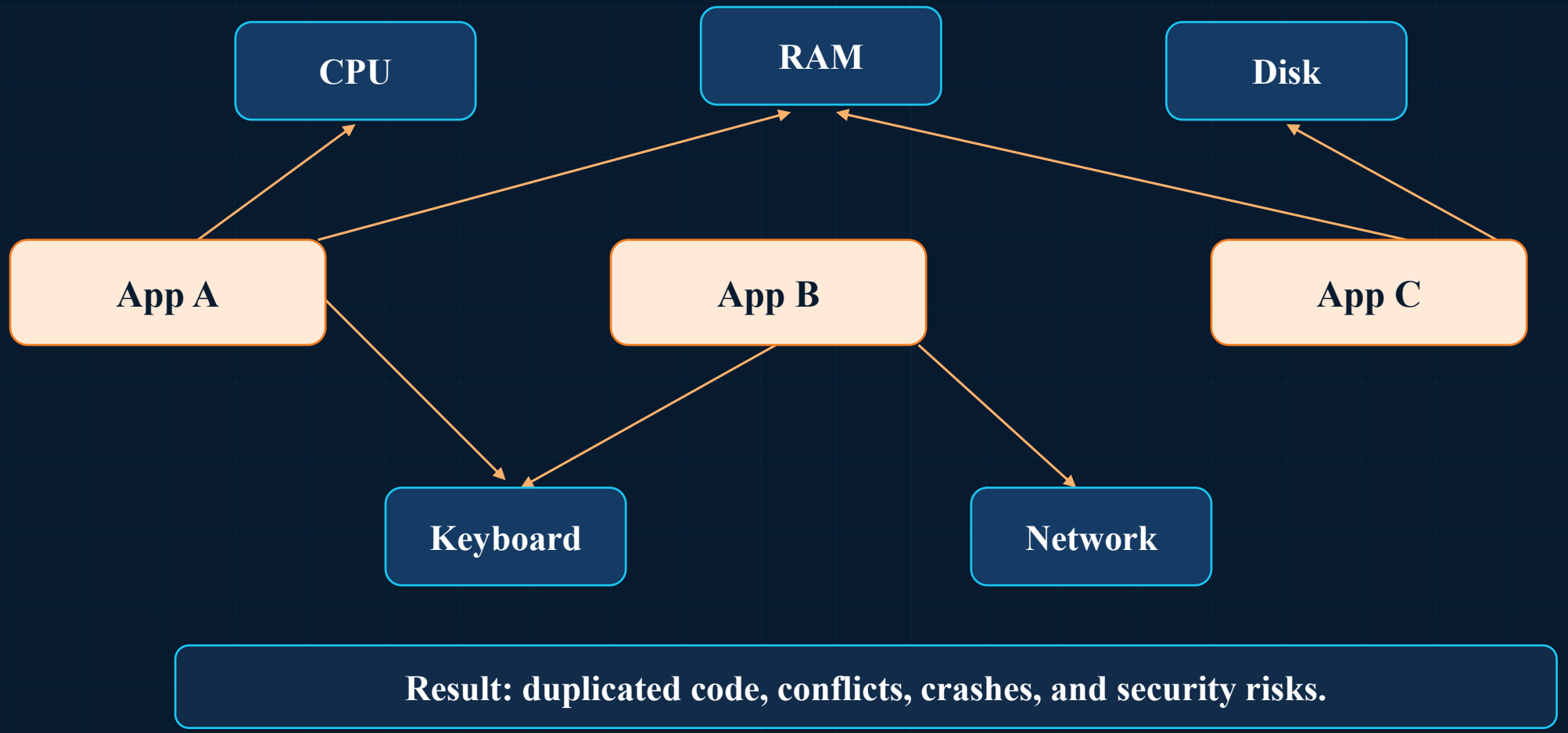
A simple action triggers a chain of operating-system services.



Today: follow this path inside the computer.

Imagine a computer without an OS

Every application would need to control hardware by itself.



What is an Operating System?

An interface between the user, applications, and computer hardware.

Interface

Makes hardware usable through common services

Resource Manager

Allocates CPU time, memory, storage, and devices

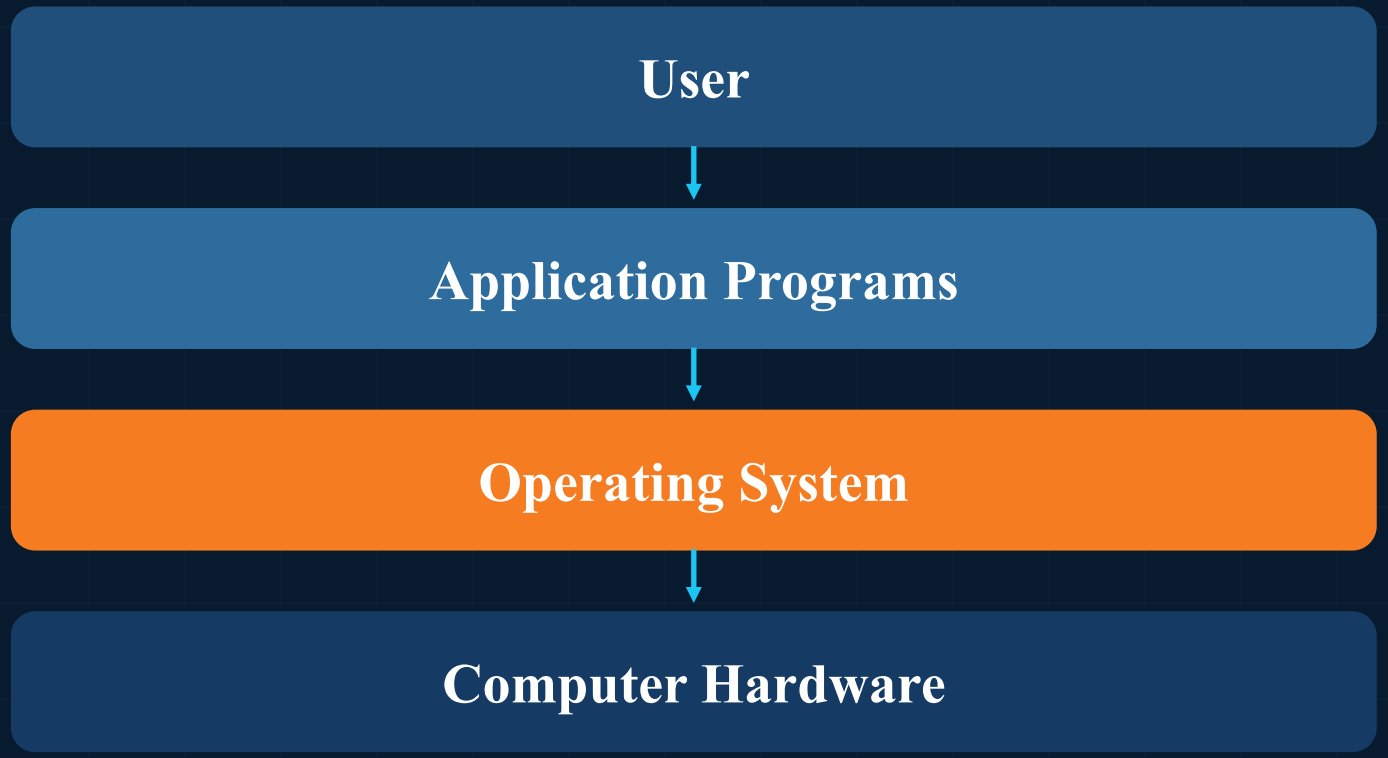
Control System

Protects the system and coordinates activity

Central message: the OS makes complex hardware manageable and safe for applications.

Where does the OS sit?

The operating system bridges applications and hardware.



Applications ask. The OS decides. Hardware executes.

Major responsibilities of an OS

The OS coordinates every shared resource in a computer.

I/O Management

Devices and data transfer

Process Management

Running programs

Memory Management

RAM allocation and protection

File Management

Organized storage

Device / Resource Control

Safe and fair access

Operating systems around us

Different devices need OS designs optimized for different environments.



Desktop / laptop	General-purpose interaction
Smartphone	Touch, sensors, battery
Server	Reliability and concurrency
Embedded / IoT	Small, specific control

01

Handling I/O



How does software communicate with physical devices?

What counts as I/O?

Input/output connects the digital system to the outside world.

Human Interface

Keyboard, mouse, camera, touchscreen

Storage

SSD, hard disk, USB drive

Communication

Wi-Fi, Ethernet, Bluetooth

Common idea: devices communicate through ports, controllers, buses, and drivers.

From application to device

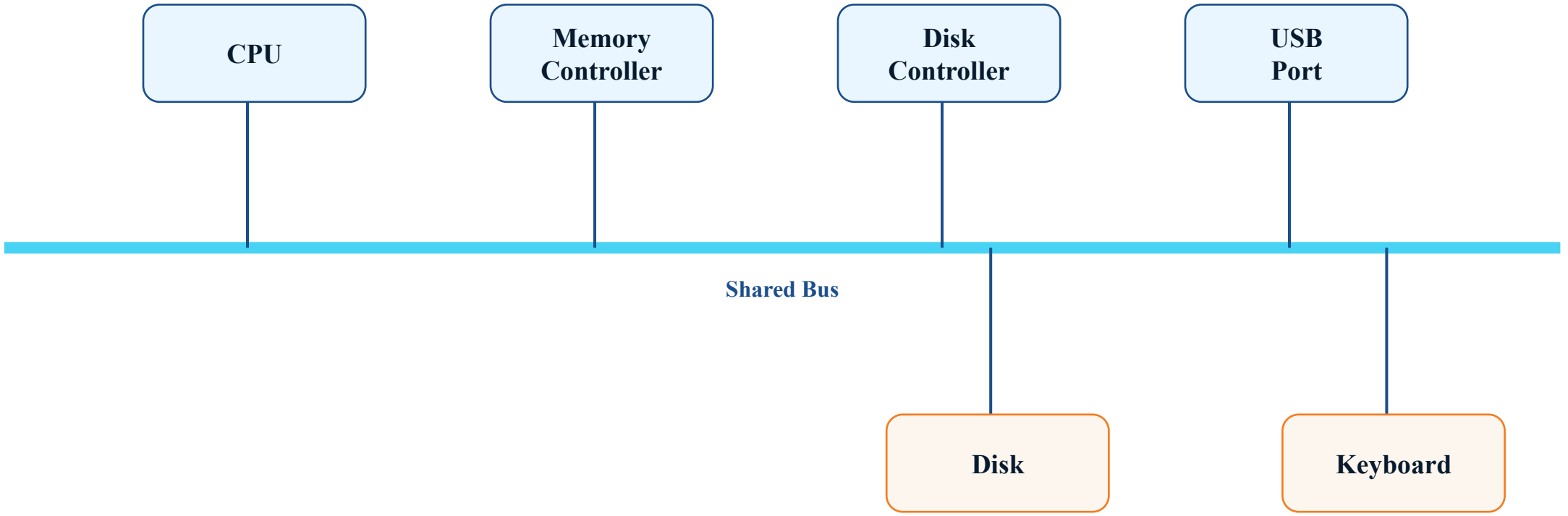
I/O requests move through layers to hide device differences.



Device-driver layer hides controller-specific details from the kernel.

Ports, controllers, and buses

Hardware communication has connection points, managers, and shared paths.



Device driver = translator

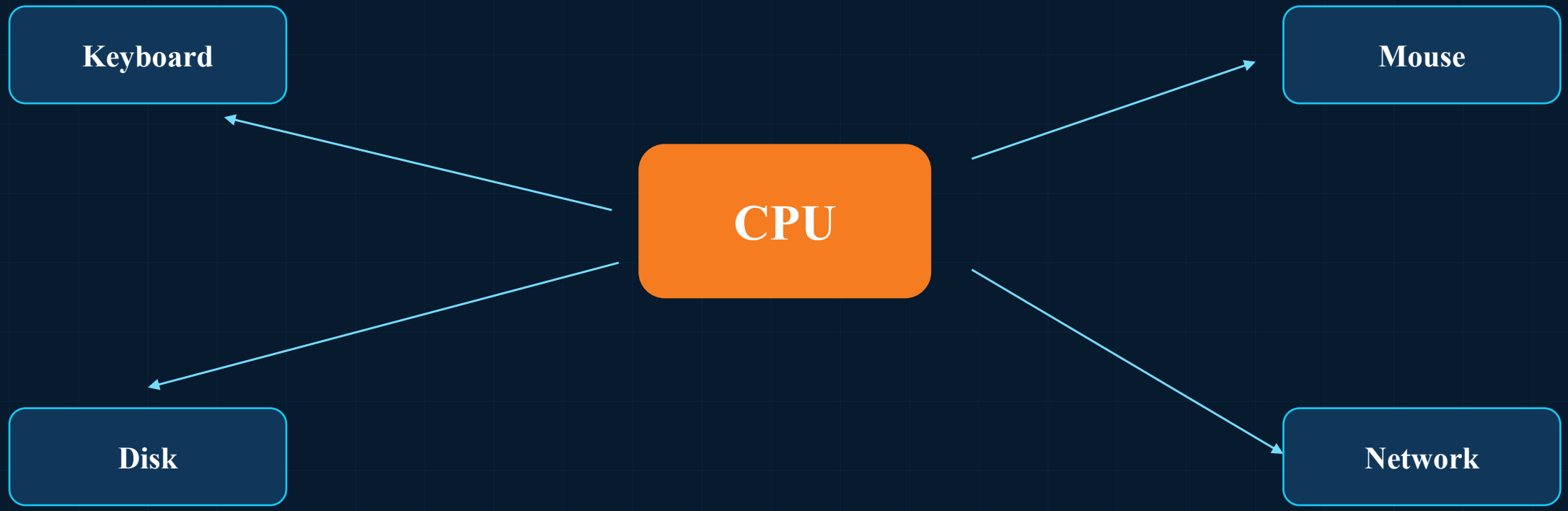
A driver converts generic OS requests into device-specific instructions.



New devices using already-implemented protocols often need less OS work.

Why polling everything is inefficient

The CPU should not waste time asking every device “Are you ready?”



Better pattern: devices signal the CPU only when attention is needed.

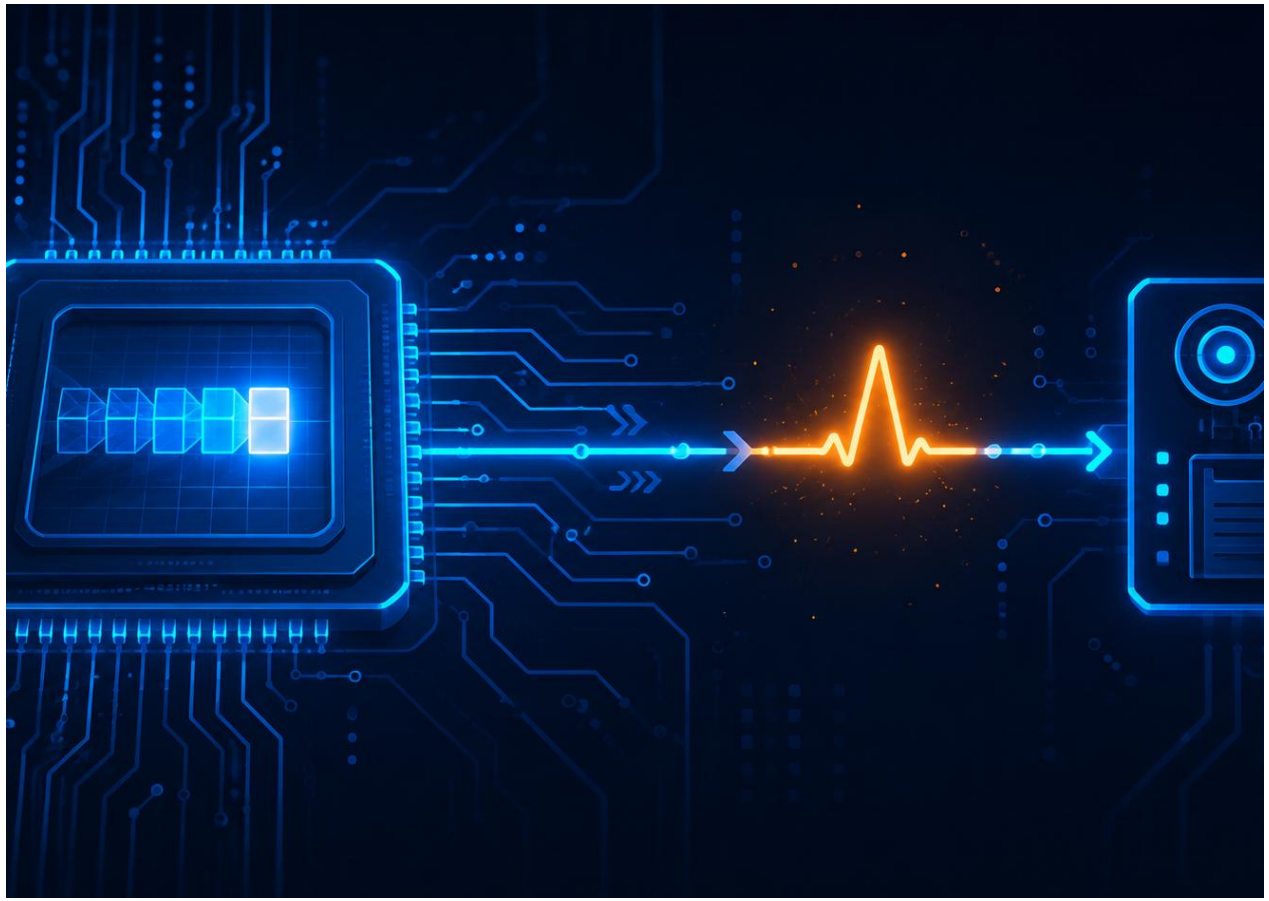
Interrupt concept

An interrupt is a signal requesting CPU attention.

CPU executes instructions

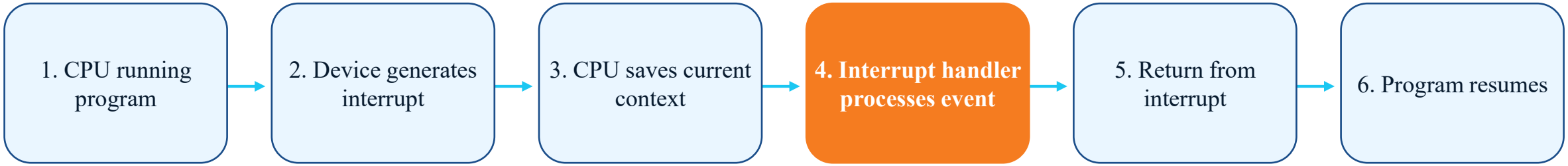
Device event occurs

Interrupt handler runs



Interrupt workflow

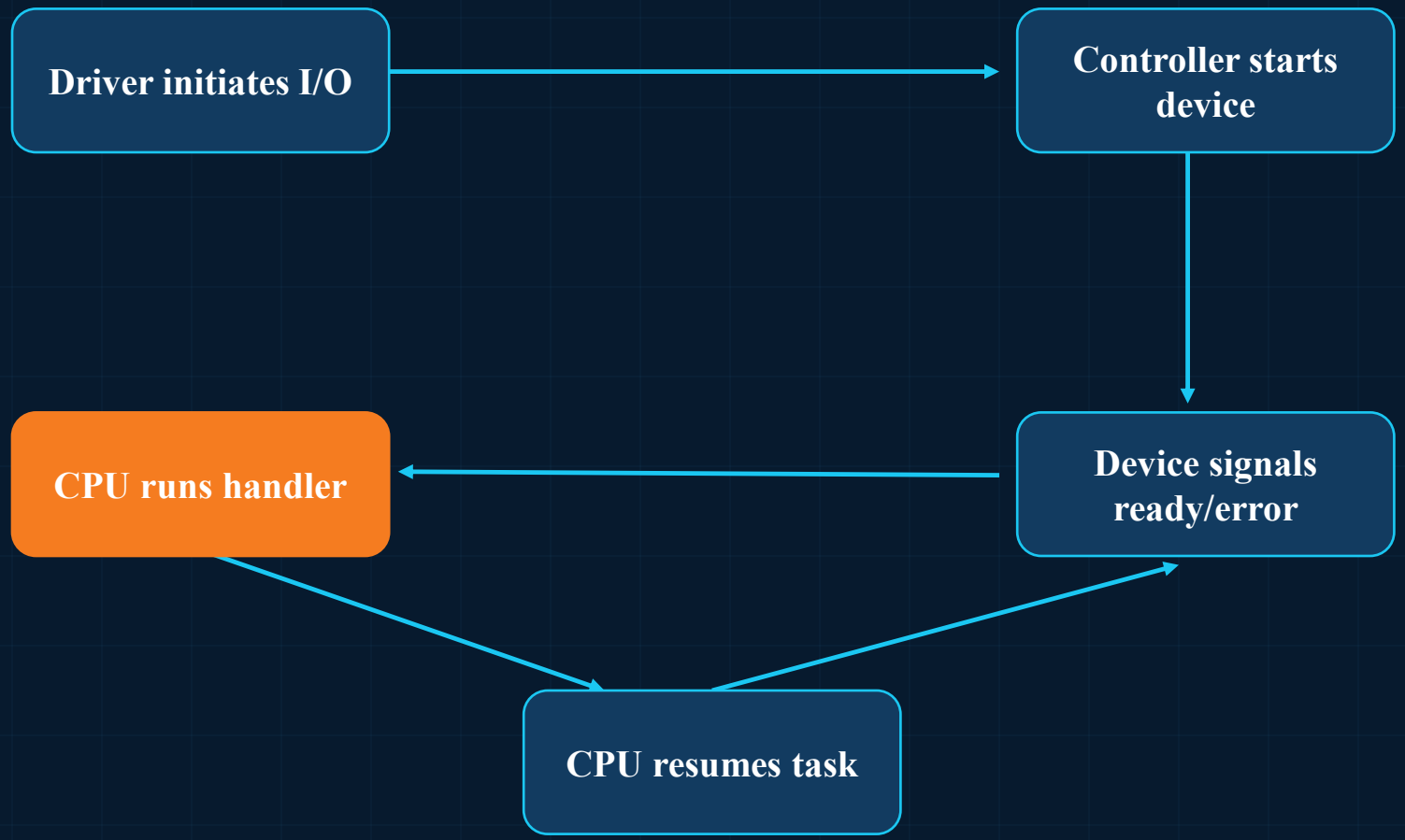
A running task is paused, a handler runs, then the task resumes.



Interrupts must be fast because they serve time-sensitive events.

Interrupt-driven I/O cycle

The device driver starts work; the device interrupts when ready.



Interrupts, exceptions, and system calls

Similar mechanism, different source of the event.

Interrupt	Exception	System Call / Trap
External hardware event	Internal execution condition	Program requests kernel service
Keyboard input, network packet	Page fault, divide error	Open file, read data

The large data problem

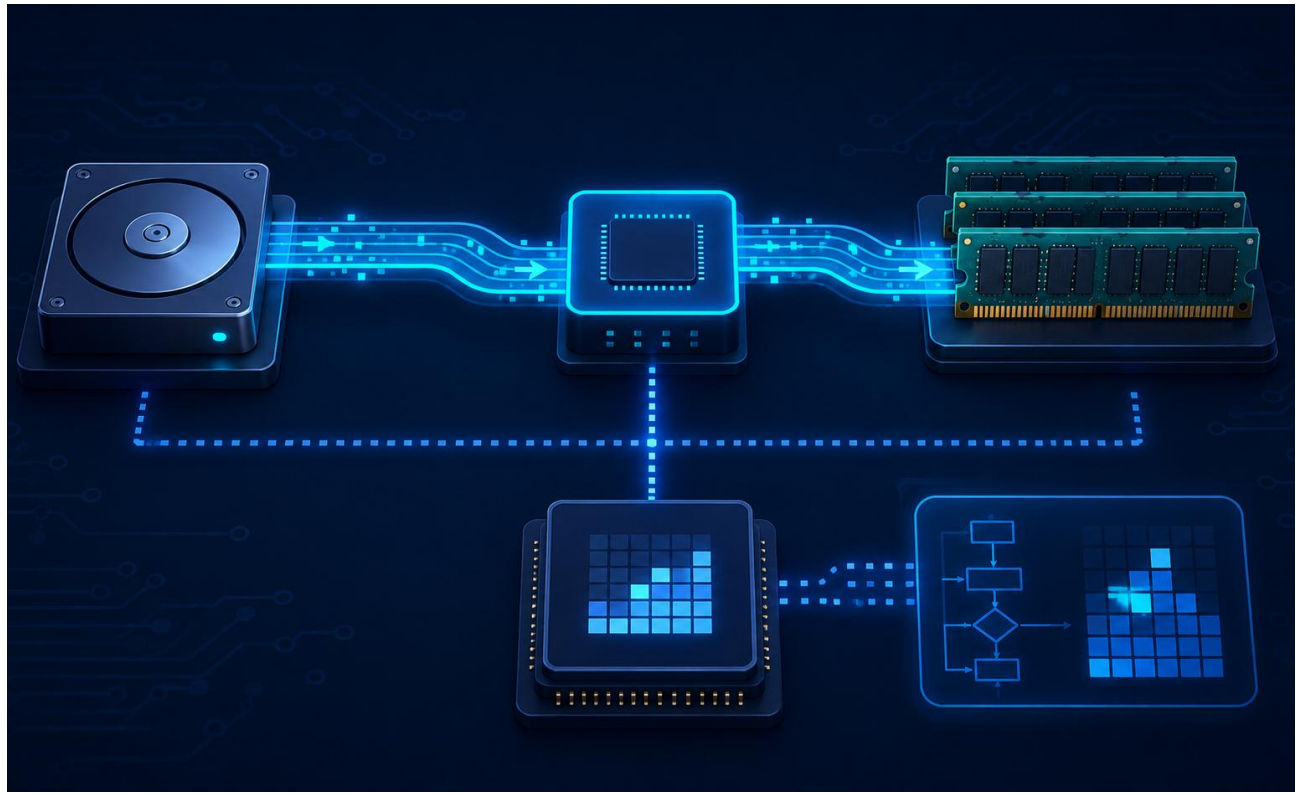
Copying every byte through the CPU would waste processing time.

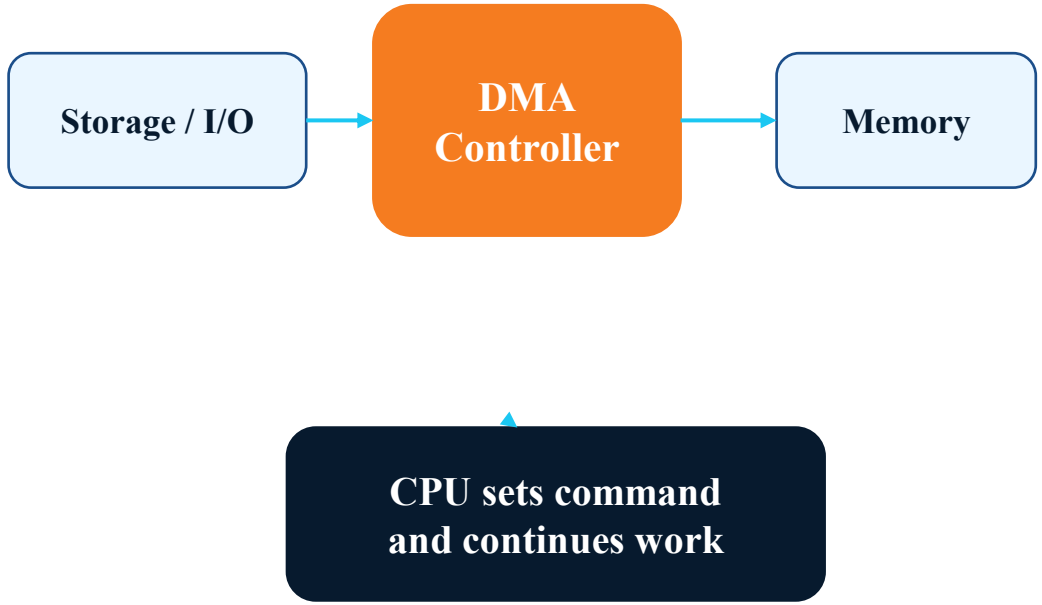


For large blocks of data, the CPU should coordinate—not carry every byte.

Direct Memory Access (DMA)

DMA transfers data directly between an I/O device and memory with minimal CPU involvement.





DMA transfer: six-step story

The OS prepares the transfer, the controller moves data, then an interrupt reports completion.

1. Request transfer
2. OS creates command block
3. DMA controller receives command
4. Controller moves bytes
5. Interrupt signals completion
6. CPU handles completion

Interrupt vs DMA

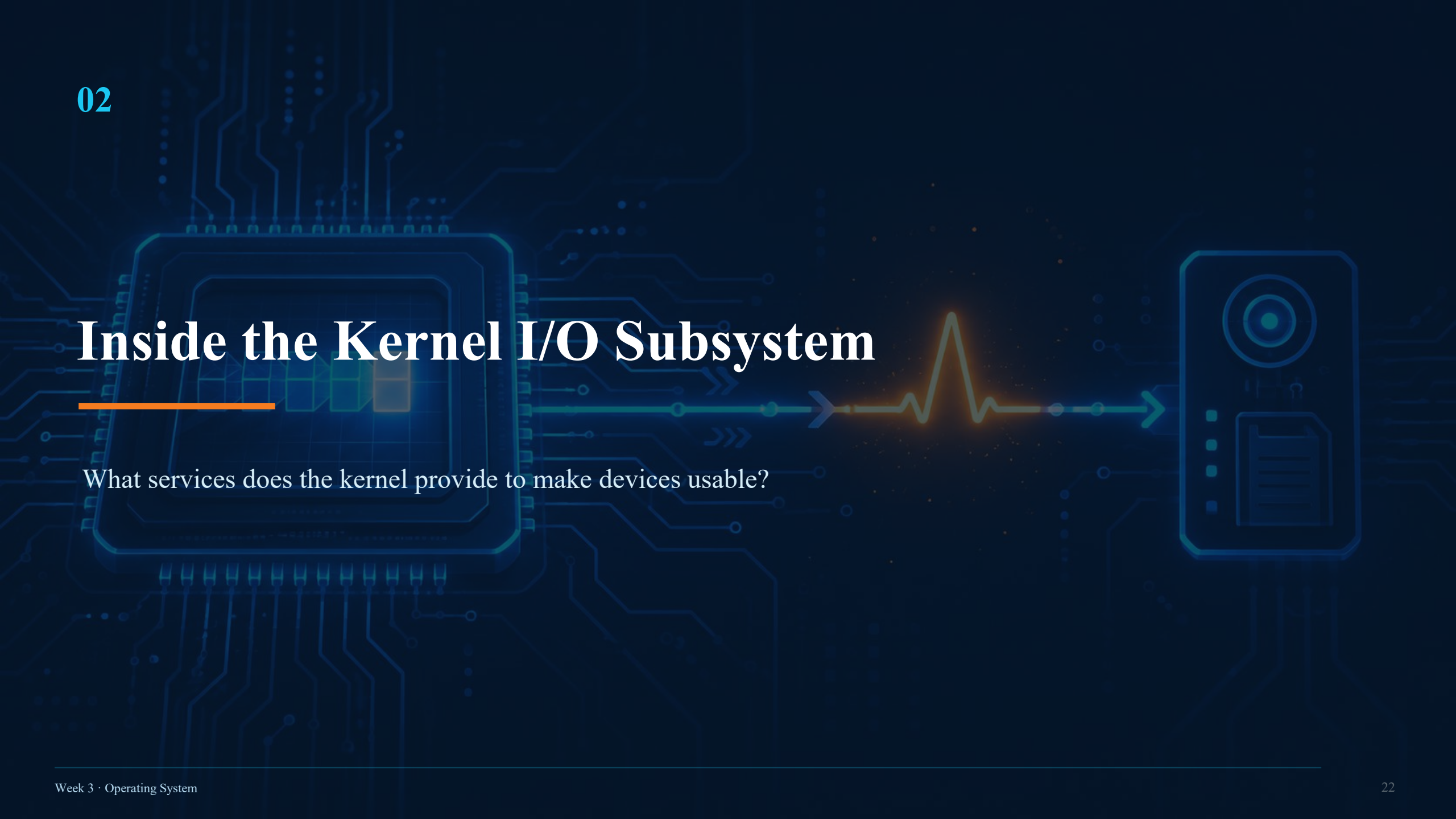
Both help I/O, but they solve different parts of the problem.

Interrupt	Small event / attention signal	Device says: “I need service”
DMA	Large block data transfer	Controller moves data with less CPU work

DMA often uses an interrupt at the end to report that the transfer is complete.

02

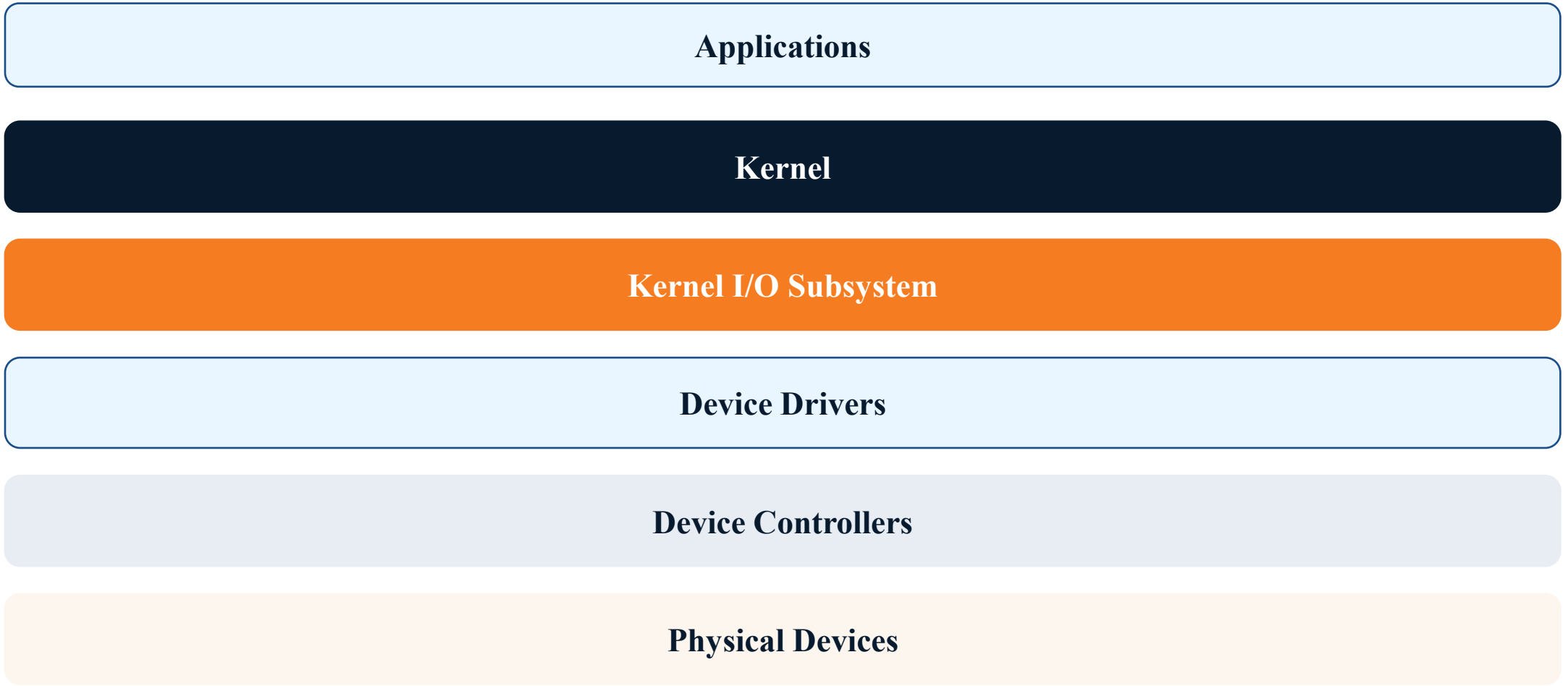
Inside the Kernel I/O Subsystem



What services does the kernel provide to make devices usable?

Kernel I/O structure

The kernel organizes software and hardware into layers.



Caching, buffering, and spooling

Kernel I/O services improve performance and coordination.

Caching

Fast copy of data

Key to performance; sometimes combined with buffering

Buffering

Temporary holding area

Smooths differences in speed between producer and consumer

Spooling

Queued output

Useful when a device serves one request at a time, such as printing

Error handling

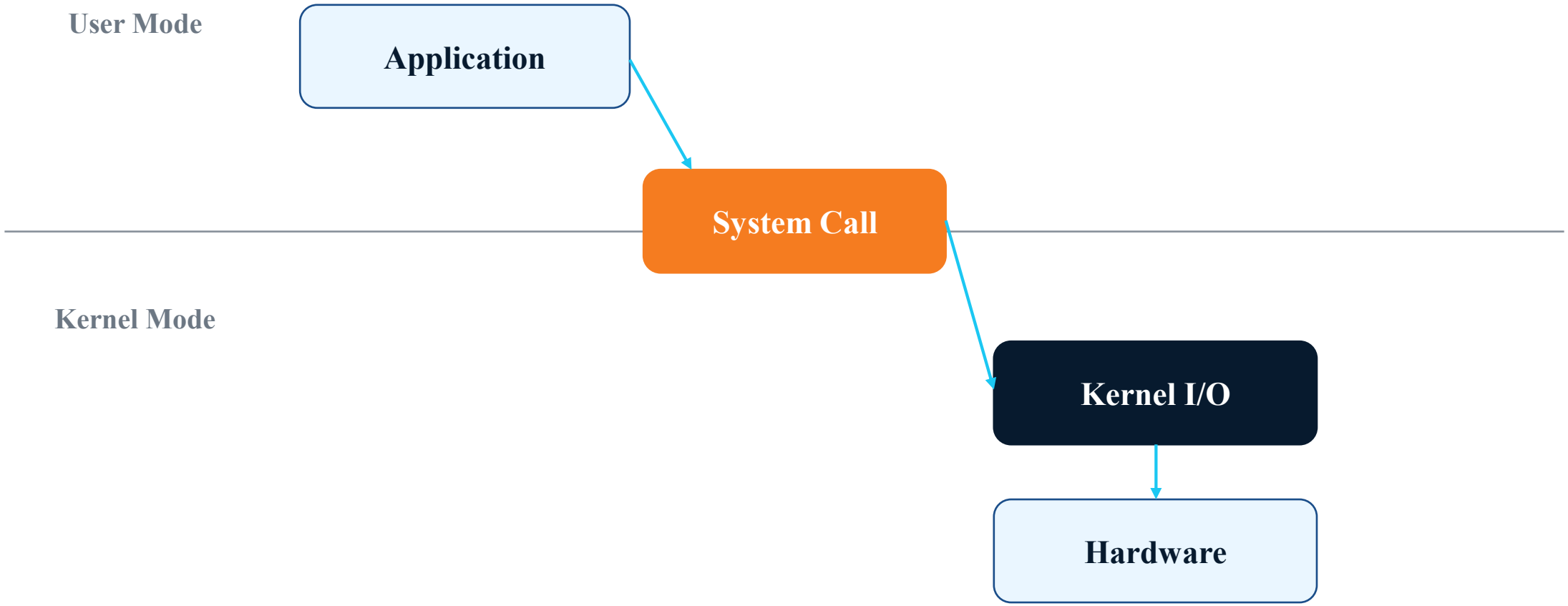
The OS detects, retries, reports, and logs device problems.



Examples: disk read failure, device unavailable, temporary write failure.

I/O protection

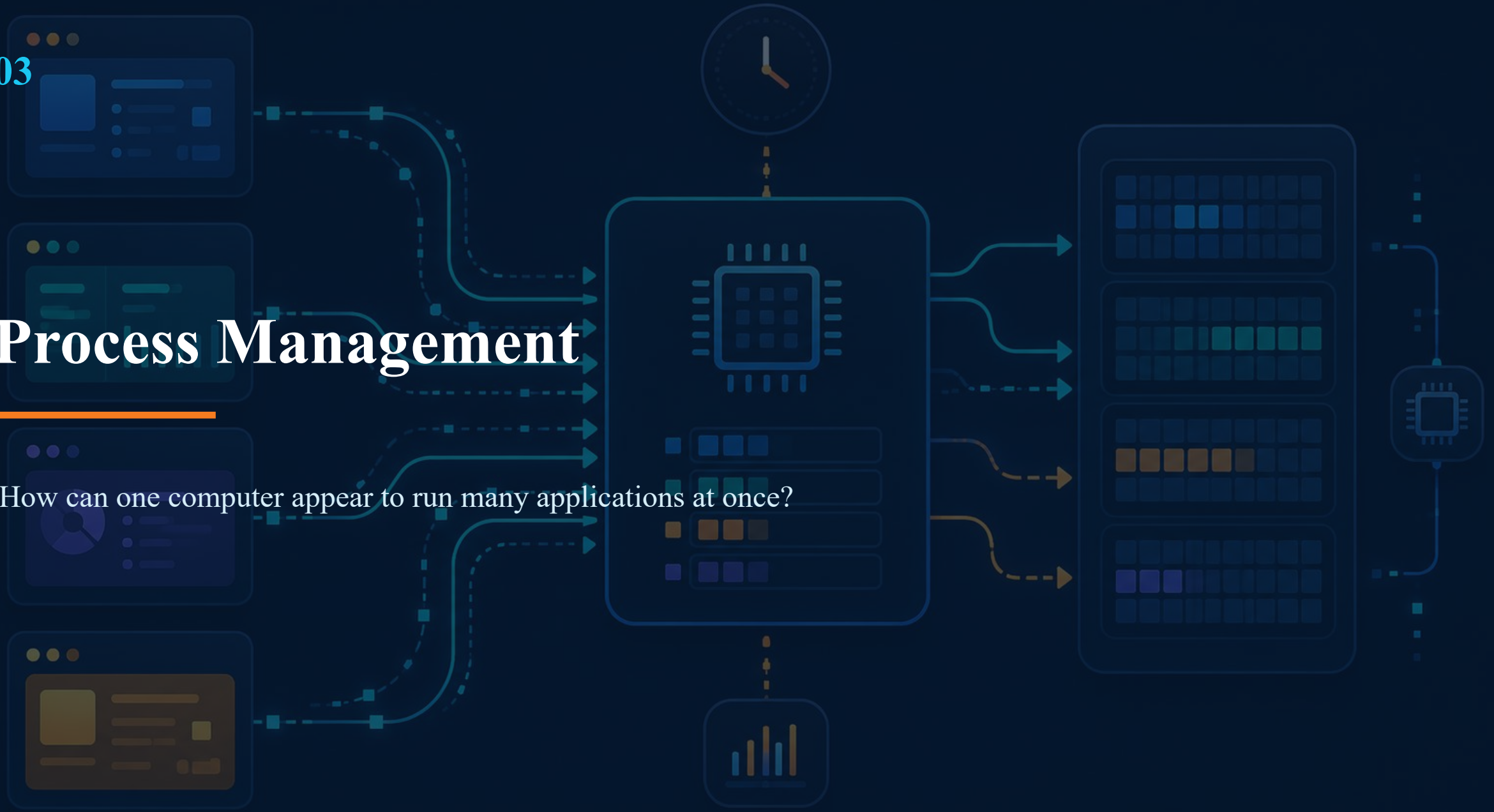
User programs must request privileged I/O through the operating system.



03

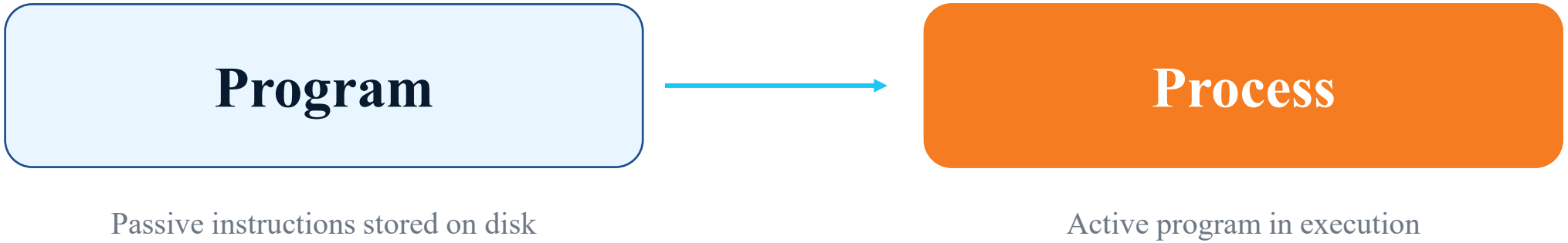
Process Management

How can one computer appear to run many applications at once?



Program vs process

A program becomes a process when it is executing.



Example: an installed browser is a program; an open browser window is a process.

What is inside a process?

A process contains code, data, runtime memory, and CPU state.

Stack — Temporary data, function calls

Heap — Dynamic memory

Data — Global variables

Text / Code — Program instructions

Runtime state

Program Counter
CPU Registers

Process states

A process changes state as it executes and waits for events.

New

being created

Ready

waiting for CPU

Running

instructions executing

Waiting

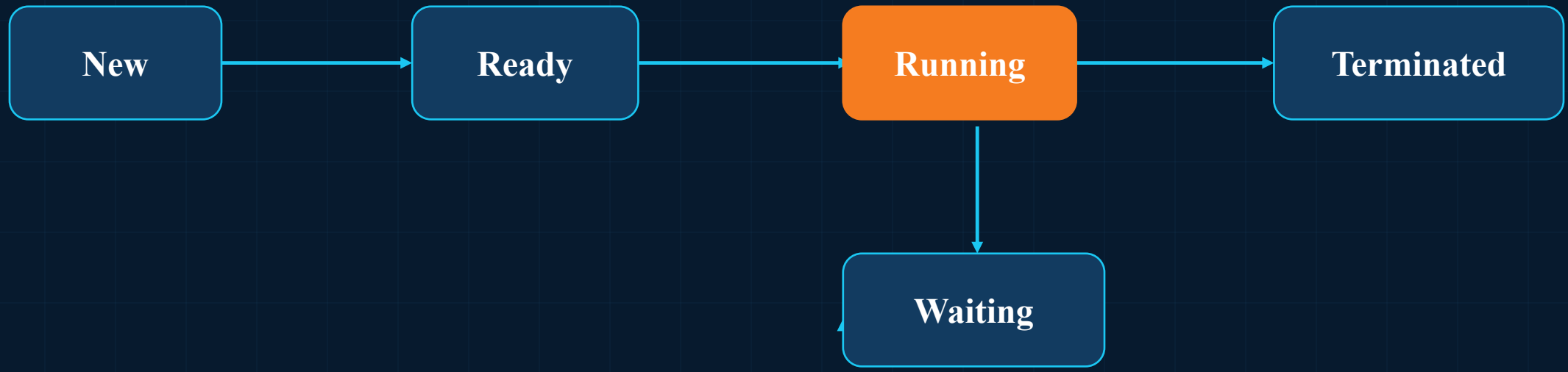
waiting for event

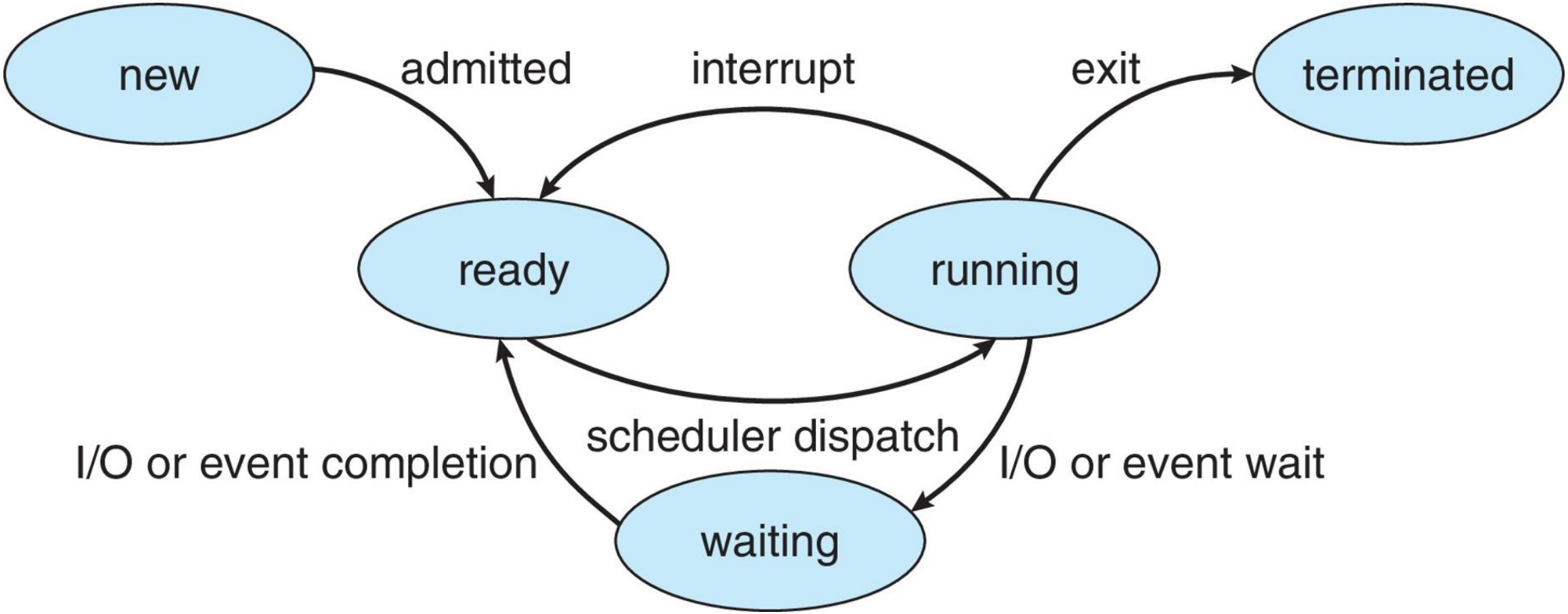
Terminated

finished execution

Process state diagram

State transitions explain how the OS controls execution.




new
admitted
interrupt
exit
terminated
ready
running
scheduler dispatch
I/O or event completion
I/O or event wait
waiting

Process Control Block (PCB)

The PCB stores what the OS needs to pause and resume a process.



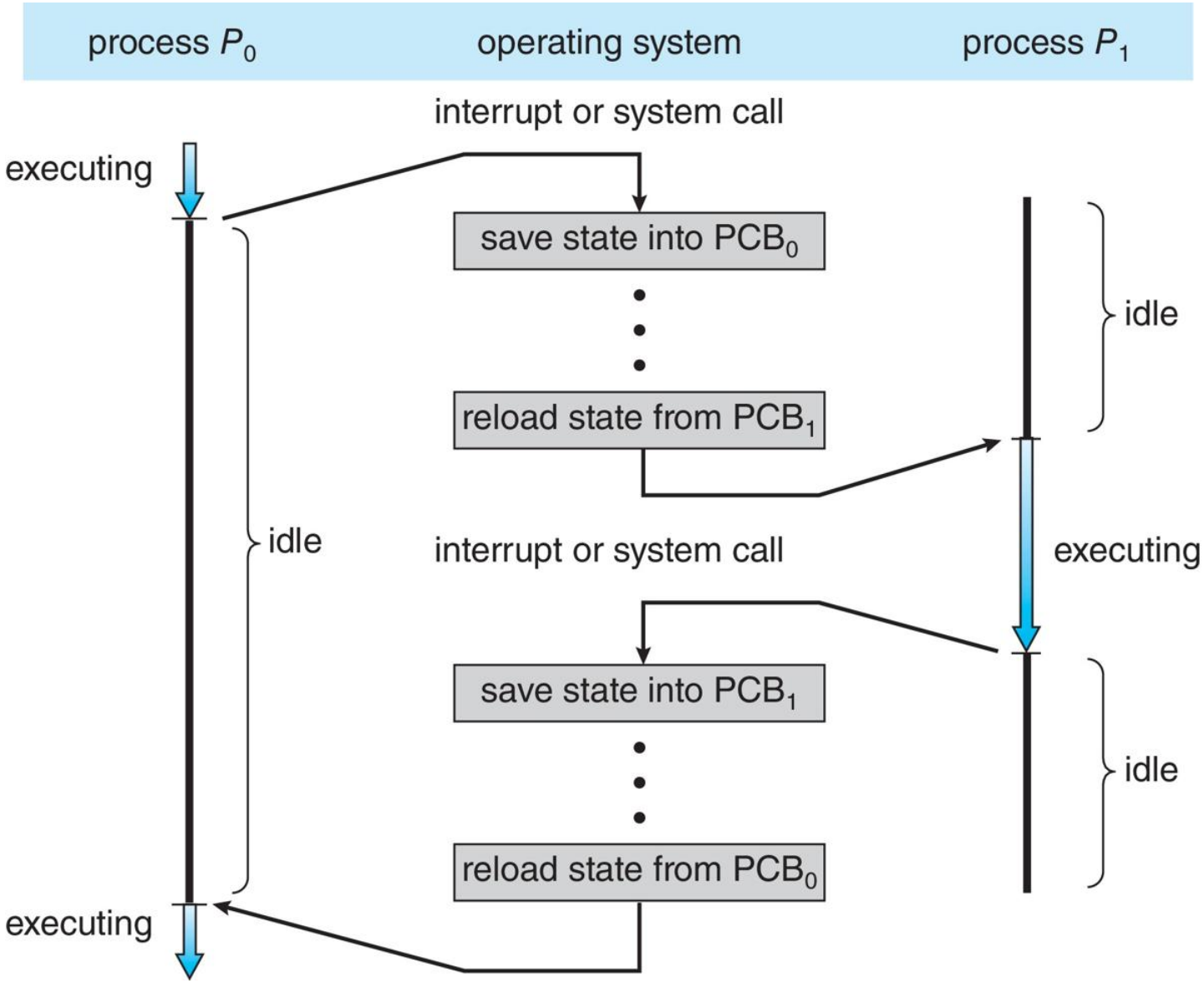
- Process state
- Program counter
- CPU registers
- Scheduling information
- Memory information
- Accounting information
- I/O status information

Context switching

The OS saves one process state and loads another.

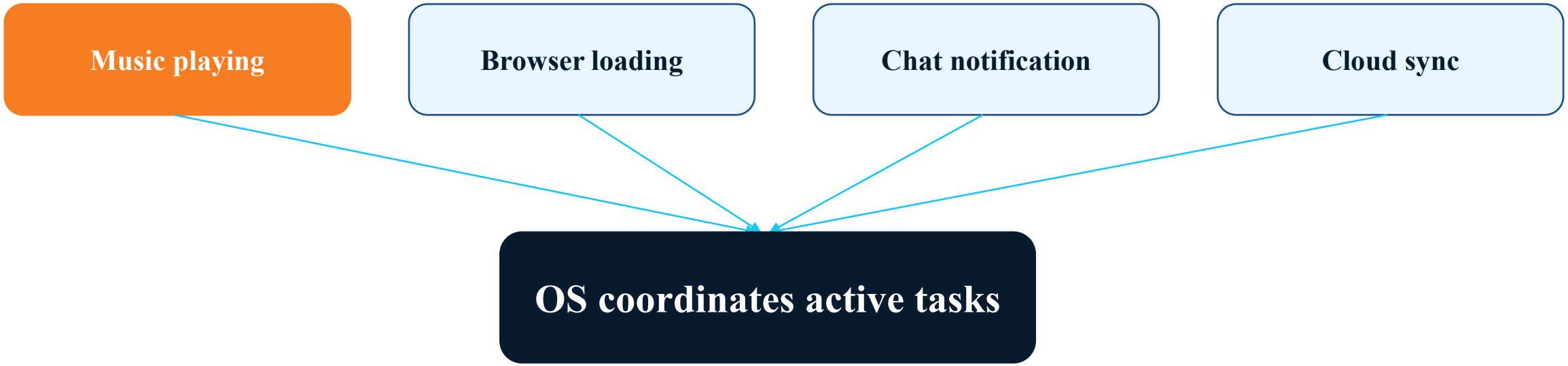


Context switching enables responsive multitasking but has overhead.


process P_0
operating system
process P_1
interrupt or system call
executing
save state into PCB_0
reload state from PCB_1
idle
idle
interrupt or system call
executing
save state into PCB_1
reload state from PCB_0
idle
executing

Why context switching matters

Everyday multitasking depends on rapid switching.



Processes can cooperate

Processes may be independent or communicate with one another.

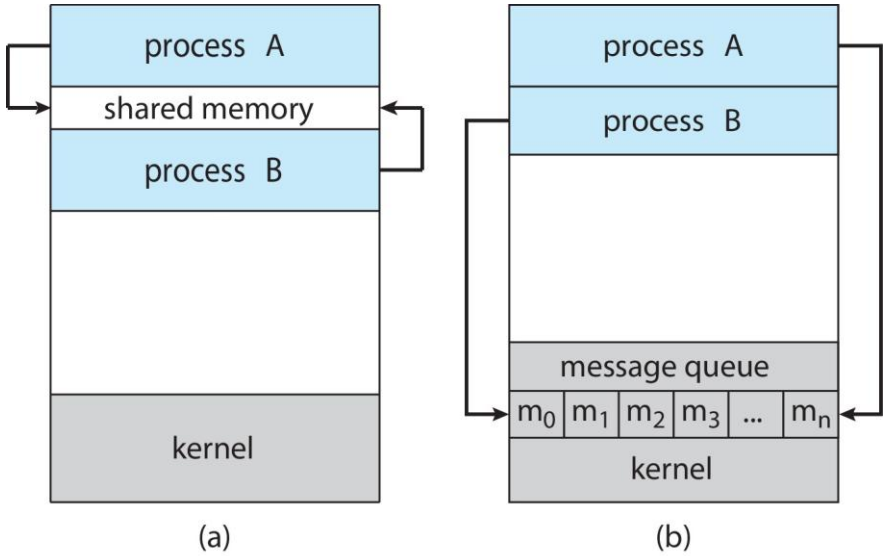


Two models: shared memory and message passing.

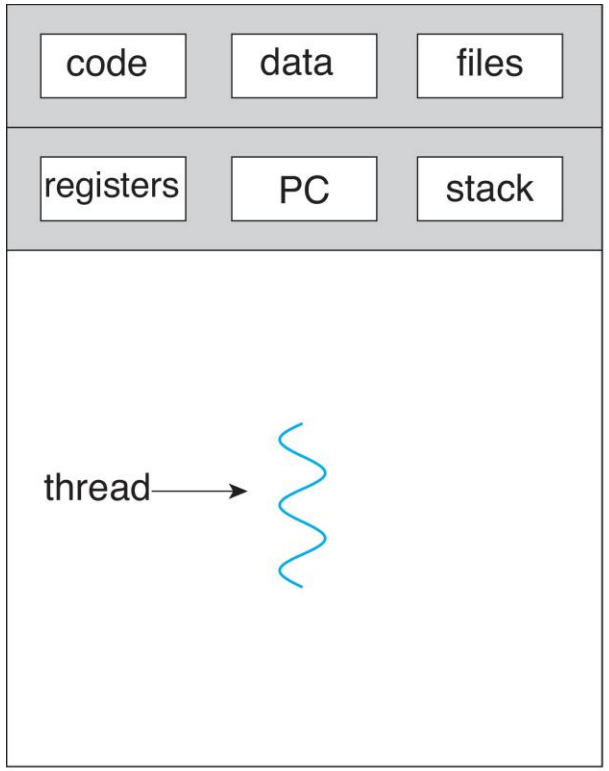
Silberschatz, Galvin and Gagne ©2018, Operating System Concepts – 10th Edition

Interprocess Communication

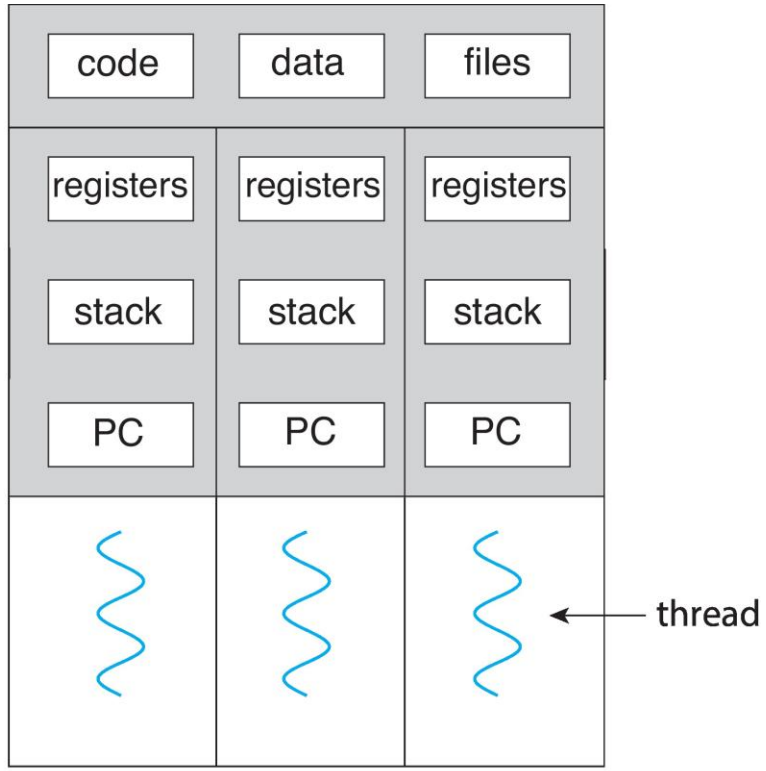
- Processes within a system may be ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**



Single and Multithreaded Processes

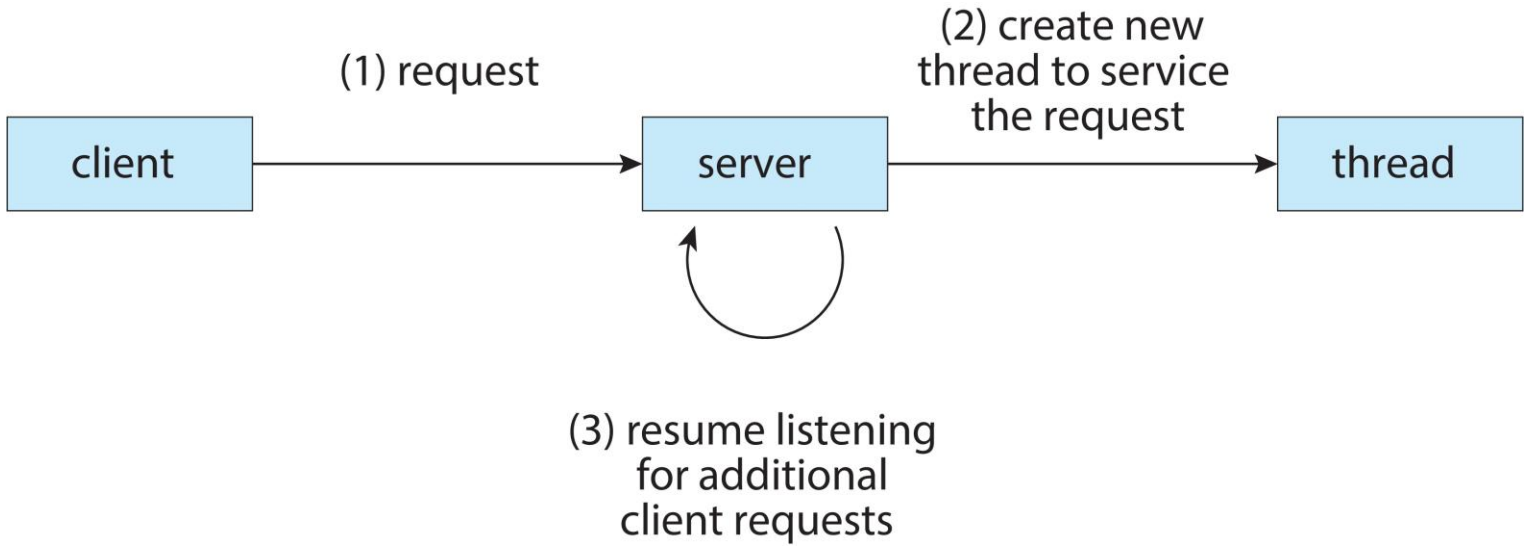


single-threaded process



multithreaded process

Multithreaded Server Architecture



Multicore Programming

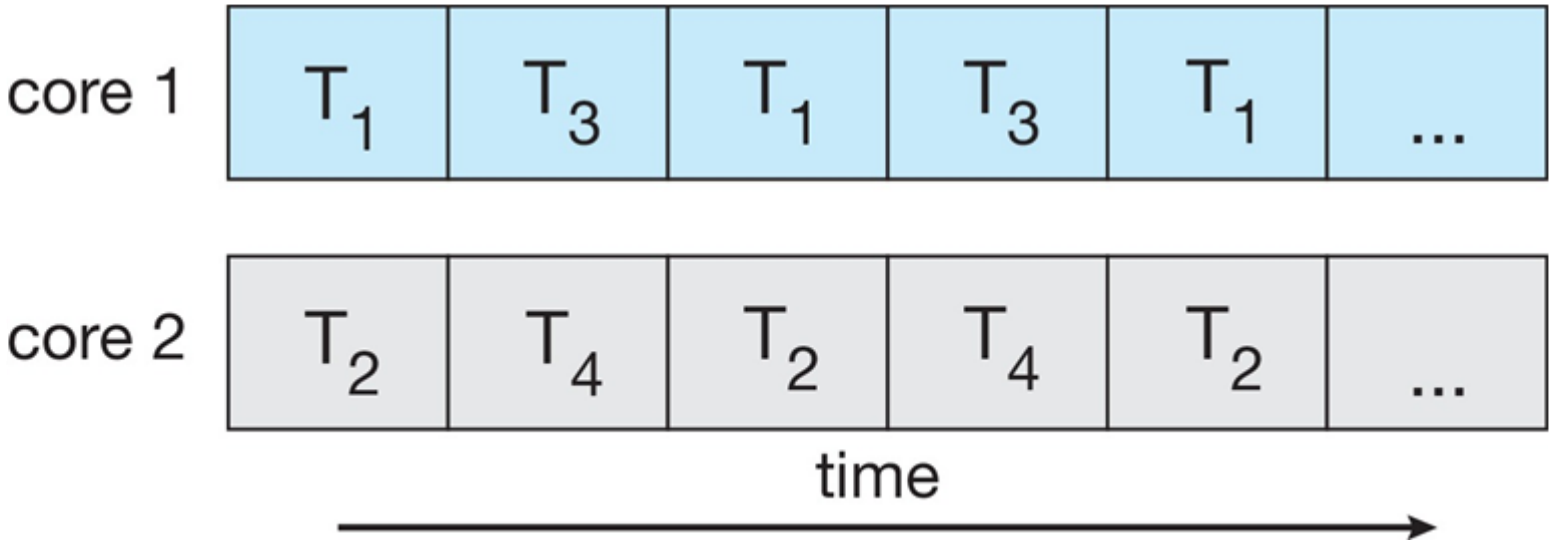
Multicore or **multiprocessor** systems puts pressure on programmers, challenges include:

Dividing activities	**Balance**	**Data splitting**	**Data dependency**	**Testing and debugging**

Parallelism implies a system can perform more than one task simultaneously

Concurrency supports more than one task making progress

Single processor / core, scheduler providing concurrency

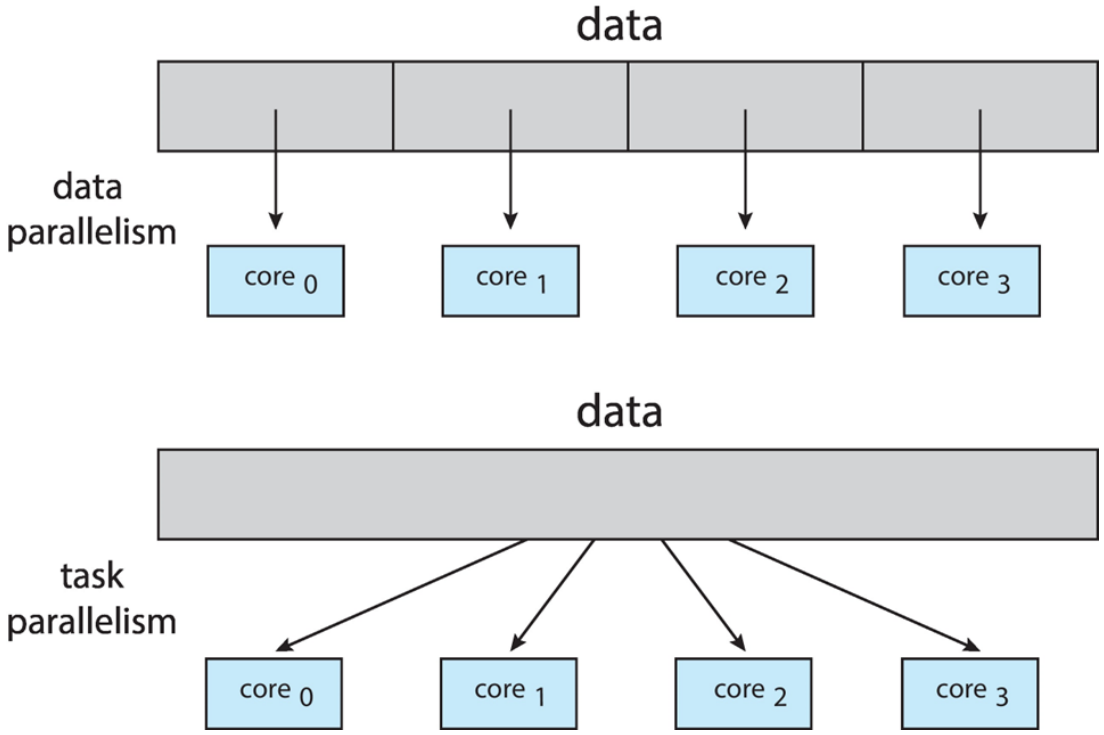


- Concurrent execution on single-core system:
- Parallelism on a multi-core system:

Silberschatz, Galvin and Gagne ©2018, Operating System Concepts – 10th Edition

Data and Task Parallelism

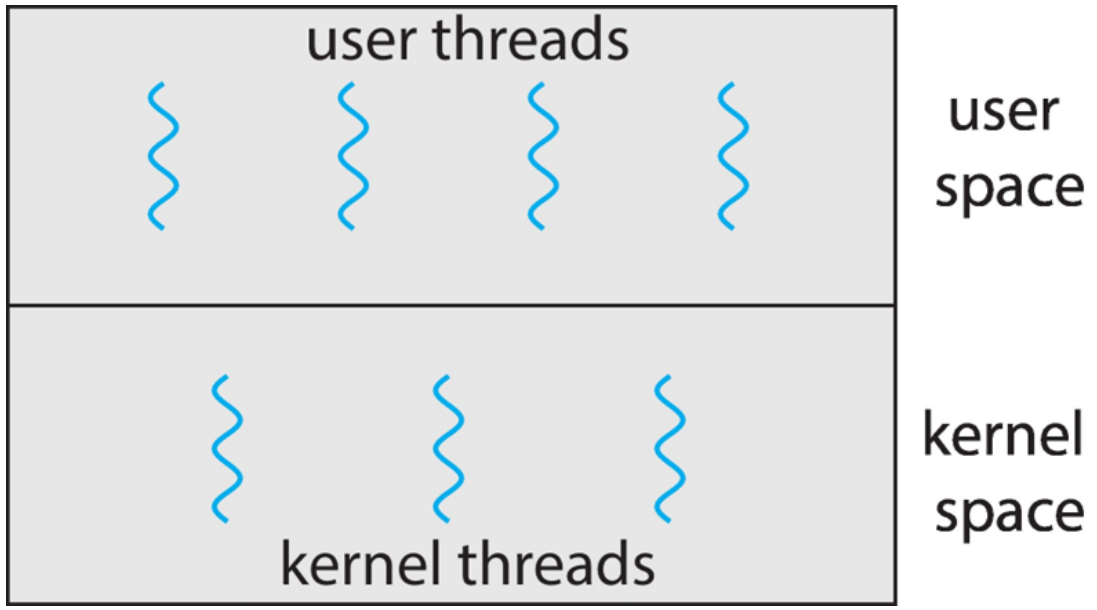
- Types of parallelism
  - **Data parallelism** – distributes subsets of the same data across multiple cores, same operation on each
  - **Task parallelism** – distributing threads across cores, each thread performing unique operation



Silberschatz, Galvin and Gagne ©2018, Operating System Concepts – 10th Edition

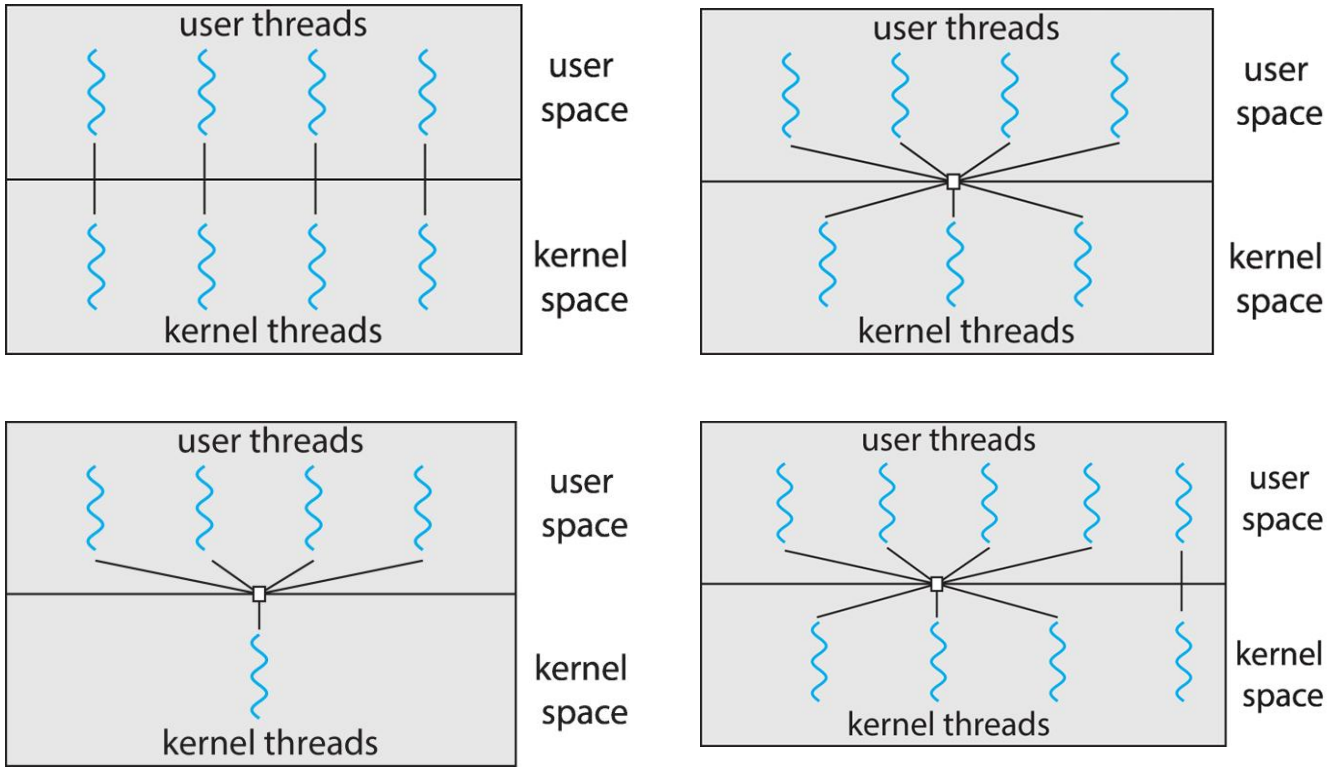
User Threads and Kernel Threads

- **User threads** - management done by user-level threads library
- Three primary thread libraries:
  - POSIX **Pthreads**
  - Windows threads
  - Java threads
- **Kernel threads** - Supported by the Kernel
- Examples – virtually all general-purpose operating systems, including:
  - Windows
  - Linux
  - Mac OS X
  - iOS
  - Android



Silberschatz, Galvin and Gagne ©2018, Operating System Concepts – 10th Edition

Multithreading Models



- One-to-One
- Many-to-One
- Many-to-Many

04

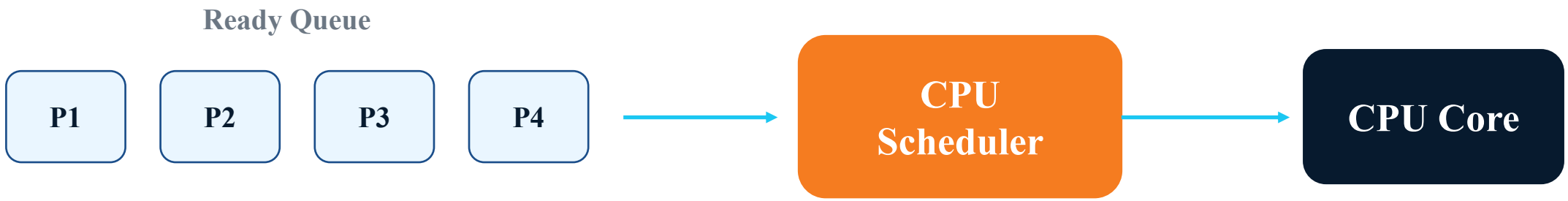
CPU Scheduling

Who gets the CPU next?



Ready queue and CPU scheduler

The scheduler selects one ready process and allocates a CPU core.



What makes a good scheduler?

Different goals can compete with one another.

CPU Utilization

keep CPU busy

Throughput

complete more processes

Turnaround Time

finish sooner

Waiting Time

reduce queue delay

Response Time

first response quickly

FCFS scheduling

First-Come, First-Served is simple but can create a convoy effect.

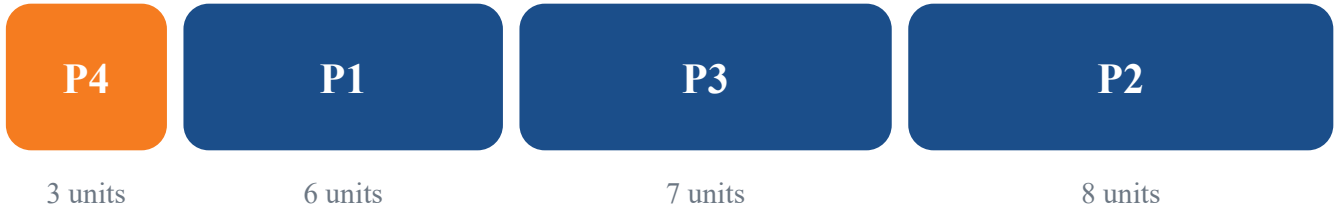
Gantt-style timeline: P1 first, then P2, then P3



Strength: easy to understand. Weakness: short processes may wait behind long ones.

Shortest-Job-First (SJF)

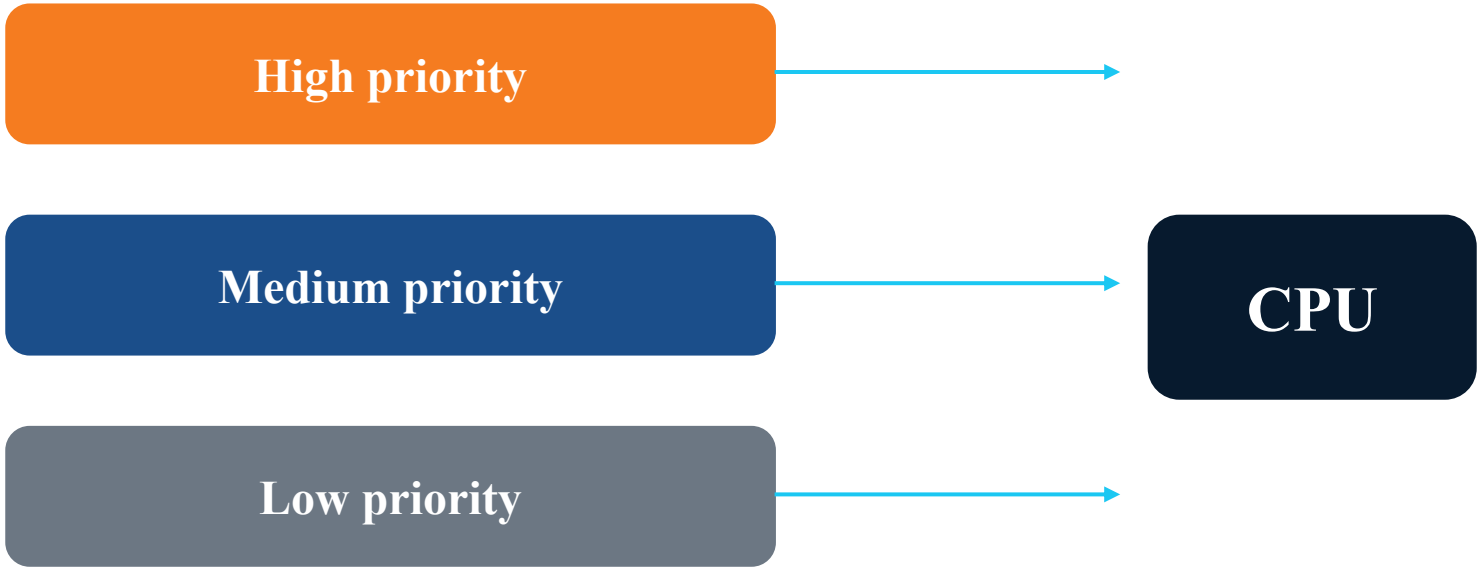
SJF gives priority to the process with the shortest next CPU burst.



Challenge: the scheduler must estimate or know the length of the next CPU burst.

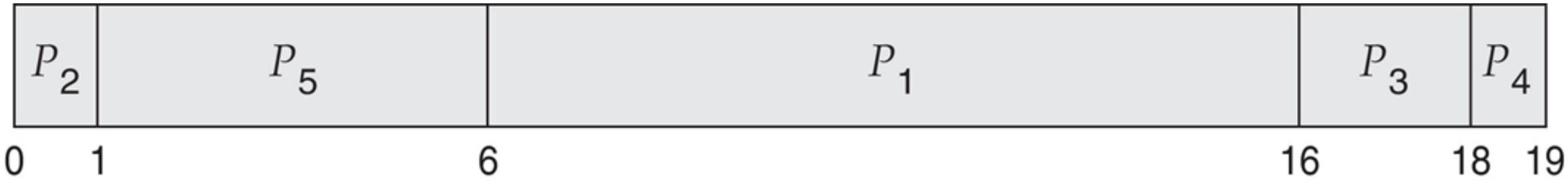
Priority scheduling

Higher-priority processes run earlier, but low-priority work can starve.



Aging can gradually increase priority to reduce starvation.

Silberschatz, Galvin and Gagne ©2018, Operating System Concepts – 10th Edition



Example of Priority Scheduling

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- Average waiting time = 8.2

Silberschatz, Galvin and Gagne ©2018, Operating System Concepts – 10th Edition



Priority Scheduling w/ Round-Robin

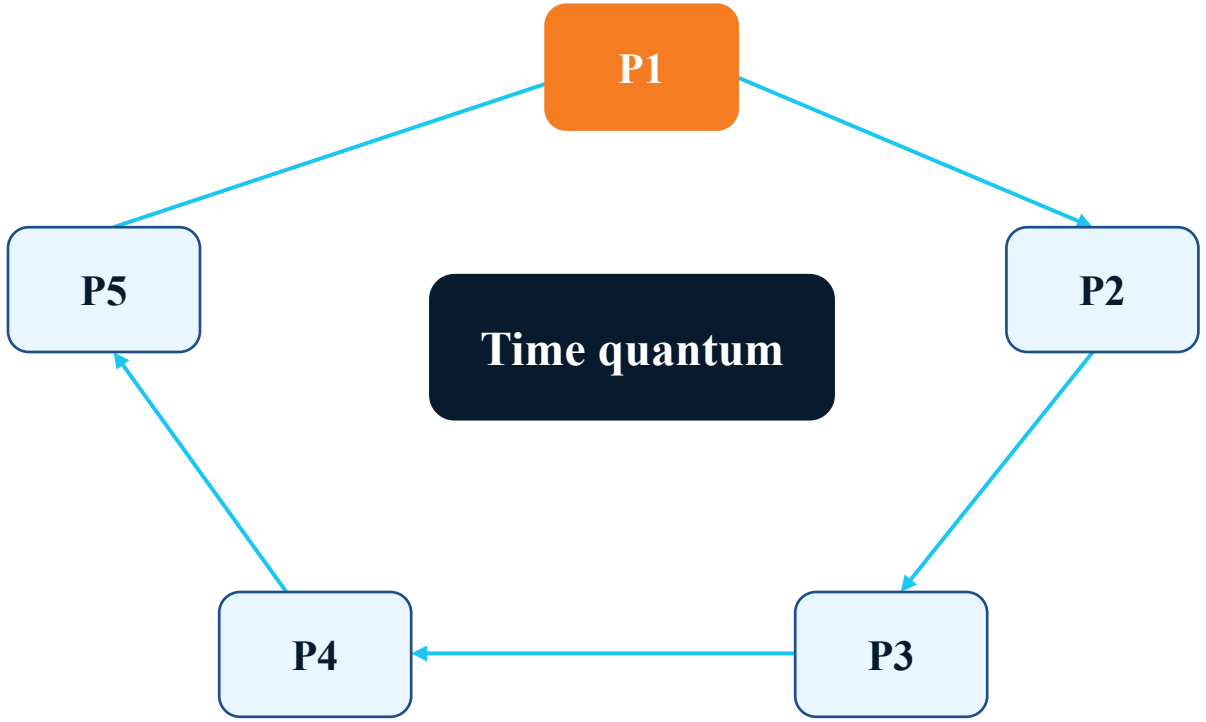
- Run the process with the highest priority. Processes with the same priority run round-robin
- Example:

Process a	Burst Time	Priority
P_1	4	3
P_2	5	2
P_3	8	2
P_4	7	1
P_5	3	3

- Gantt Chart with time quantum = 2

Round Robin scheduling

Each ready process receives a short time slice in rotation.



Good for interactive systems because no ready process waits forever for a first response.

Scheduling strategies at a glance

Each strategy makes a different tradeoff.

Algorithm	Basic idea	Strength	Weakness
FCFS	Queue order	Simple	Convoy effect
SJF	Shortest burst	Low average waiting	Needs estimates
Priority	Priority value	Handles urgency	Starvation risk
Round Robin	Time slices	Responsive	Context-switch overhead

Mini activity: schedule the queue

Predict how the timeline changes under different algorithms.

Process	Arrival	Burst
P1	0	6
P2	1	3
P3	2	4
P4	3	2

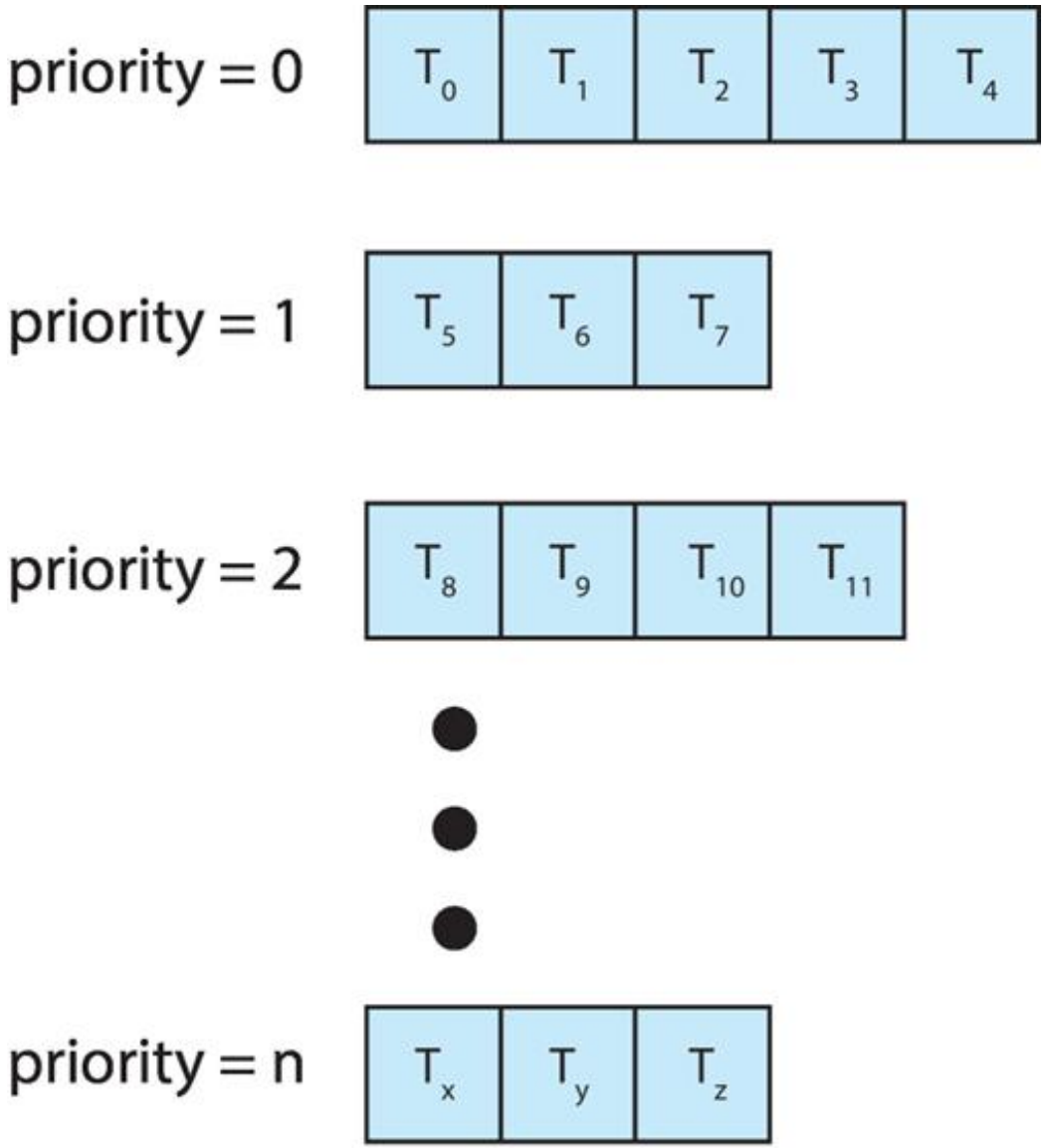
Questions

1. How would FCFS order these processes?
2. Which process would SJF choose first after P1?
3. How might Round Robin improve response?

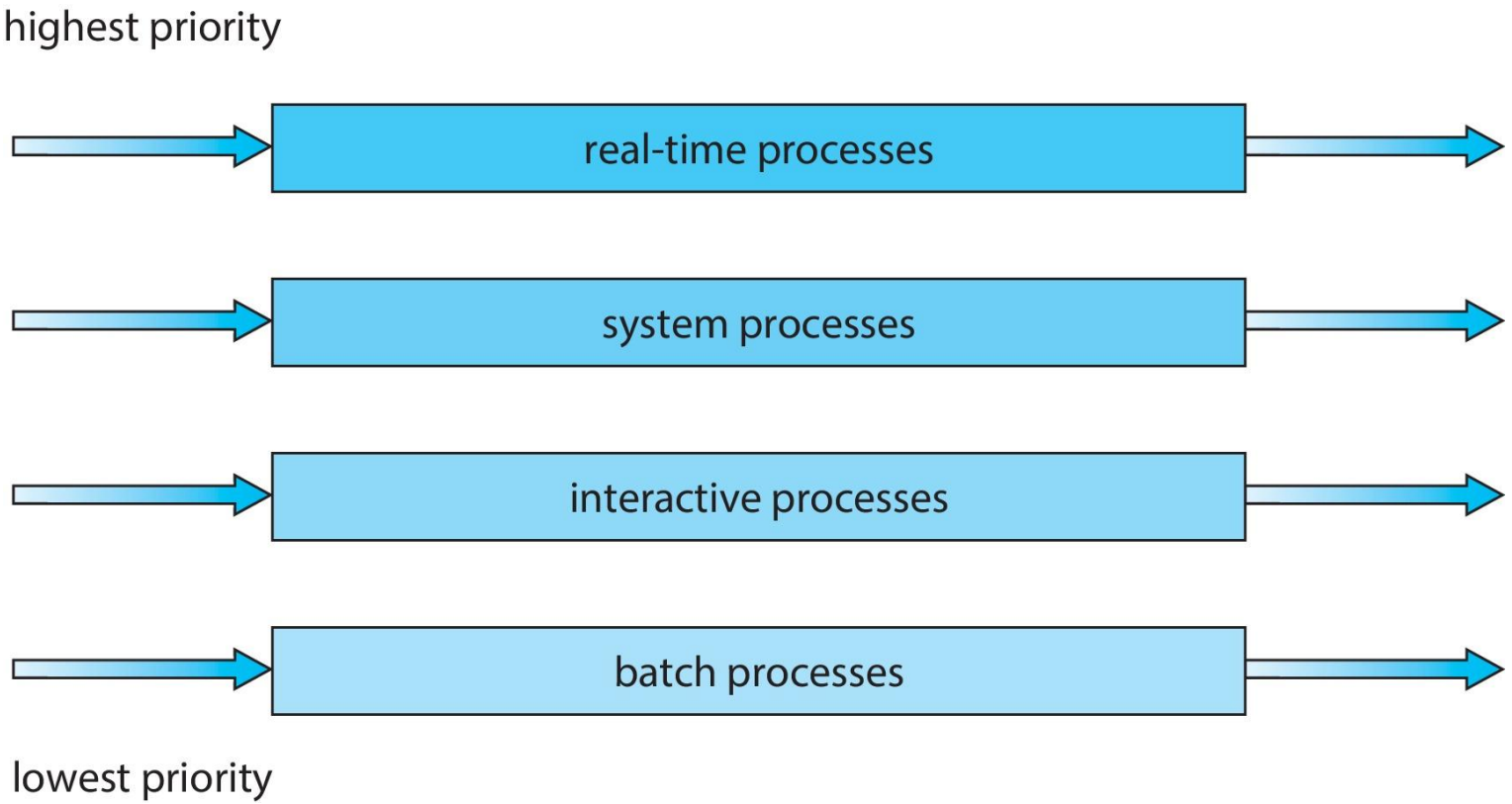
Silberschatz, Galvin and Gagne ©2018, Operating System Concepts – 10th Edition

Multilevel Queue

- The ready queue consists of multiple queues
- Multilevel queue scheduler defined by the following parameters:
  - Number of queues
  - Scheduling algorithms for each queue
  - Method used to determine which queue a process will enter when that process needs service
  - Scheduling among the queues



Multilevel Queue



05

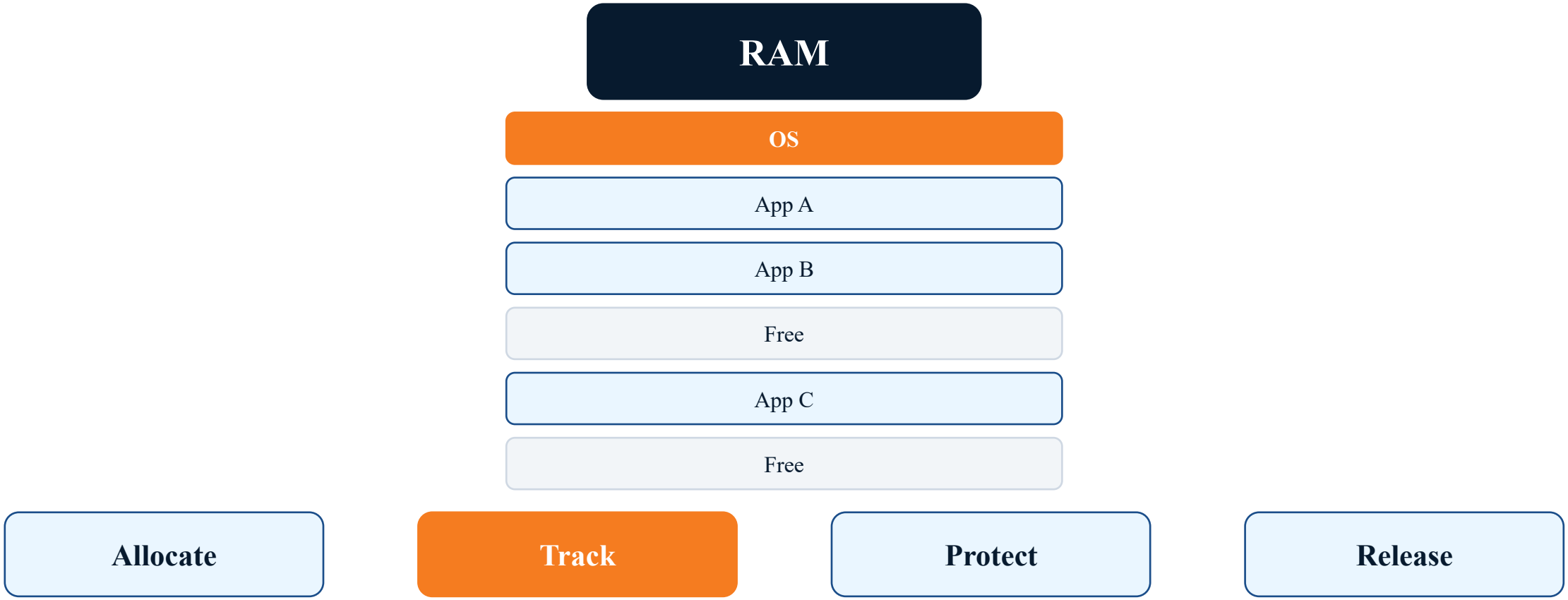
Memory Management

How can many programs safely share RAM?



Why memory management matters

RAM must be allocated, tracked, protected, and released.



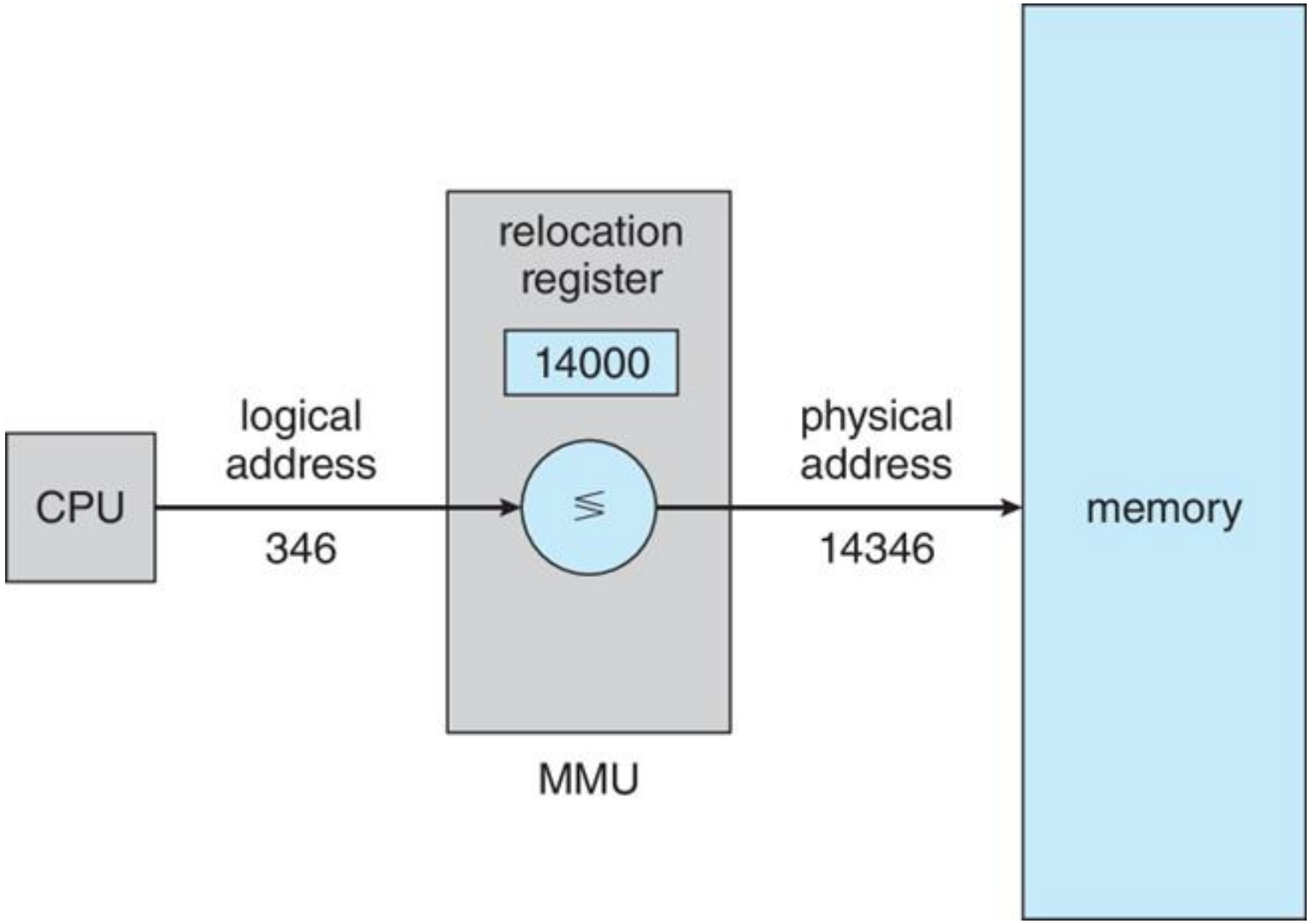
Logical vs physical address

Programs see logical addresses; memory hardware uses physical addresses.



This separation is central to memory management and protection.

Memory-Management Unit (MMU)



Contiguous allocation idea

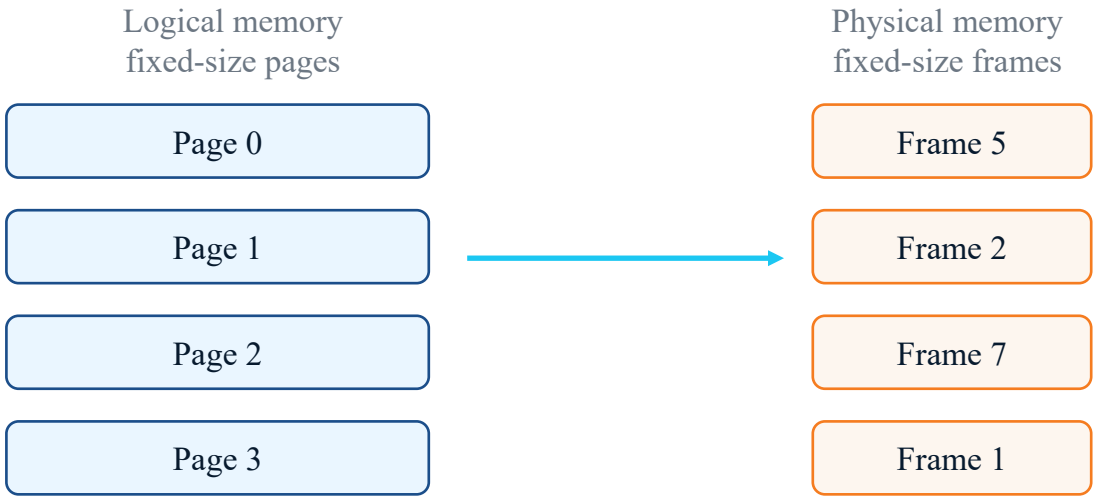
A base and limit can protect one process from accessing another area.

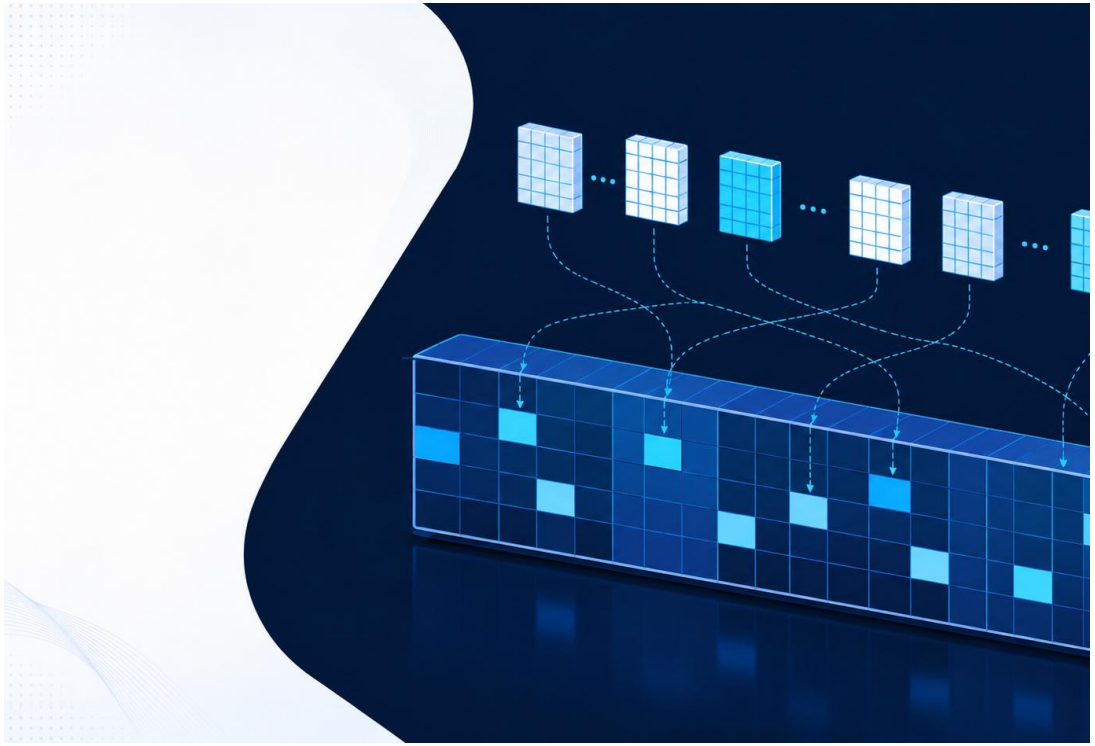


The MMU maps logical addresses dynamically while enforcing bounds.

Paging

Pages and frames let a process occupy non-contiguous physical memory.





Page table

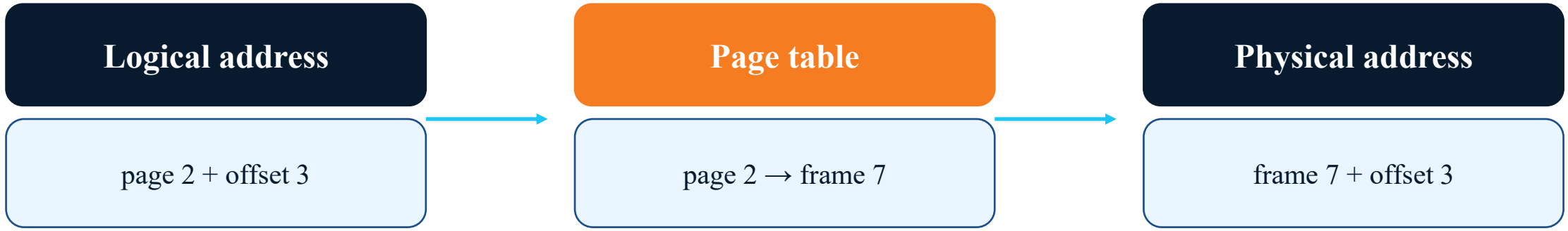
A page table maps logical pages to physical frames.

Page	Frame
0	5
1	2
2	7
3	1

Page number indexes the page table. The offset stays the same inside the frame.

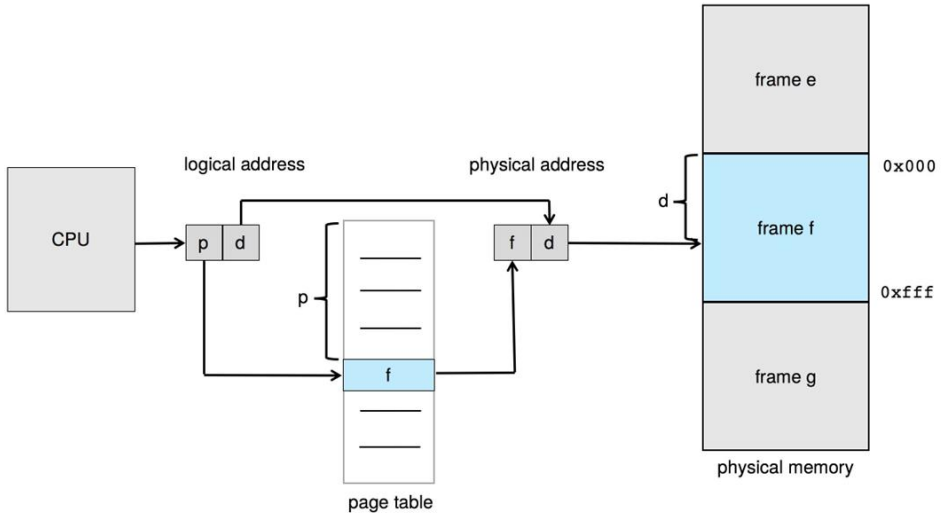
Paging walkthrough

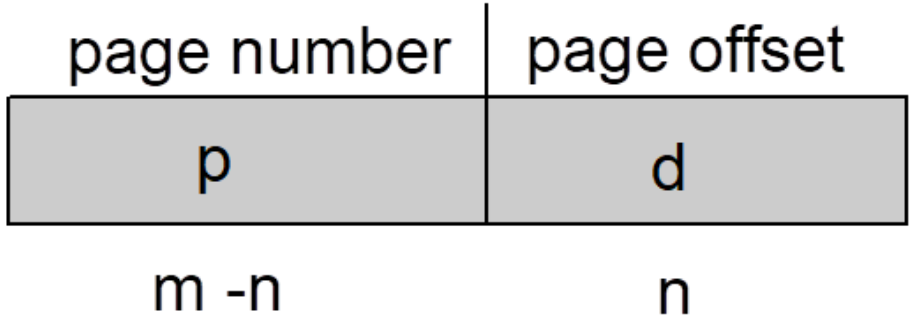
Illustrative example: logical address becomes frame plus offset.



Numbers are illustrative for teaching, not copied from a real machine.

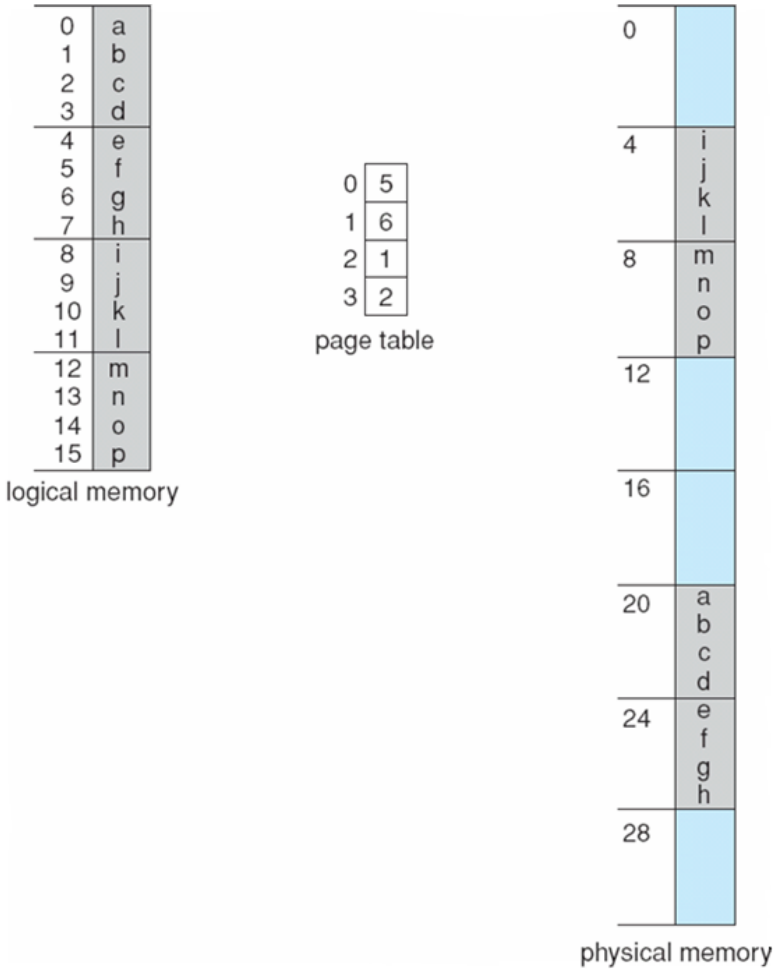
Paging (2)





- Address generated by CPU is divided into:
  - **Page number** (***p***) – used as an index into a **page table** which contains base address of each page in physical memory
  - **Page offset** (***d***) – combined with base address to define the physical memory address that is sent to the memory unit
  - For given logical address space 2^m and page size 2^n

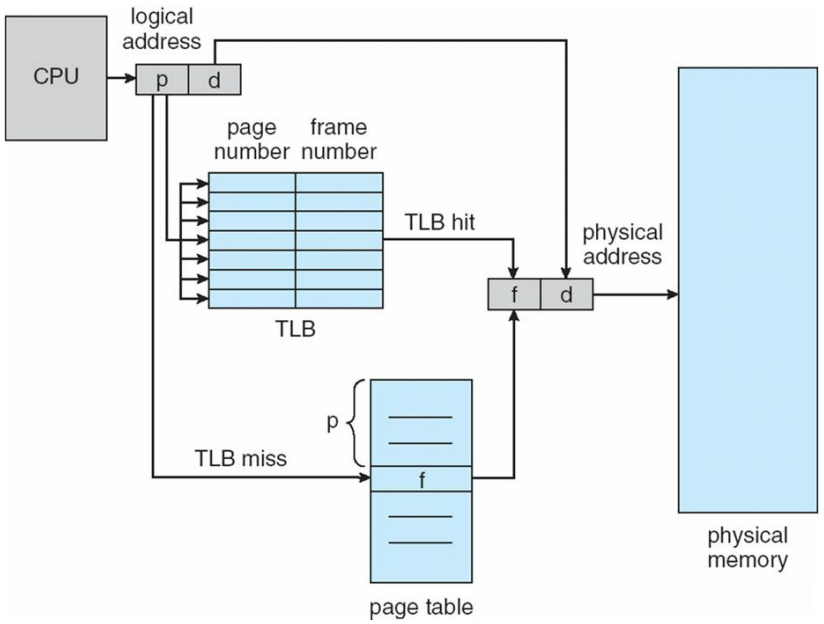
Silberschatz, Galvin and Gagne ©2018, Operating System Concepts – 10th Edition



Paging Example

- Logical address: n = 2 and m = 4. Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages)

Silberschatz, Galvin and Gagne ©2018, Operating System Concepts – 10th Edition



Translation Look-Aside Buffer (TLB)

- Some TLBs store **address-space identifiers** (**ASIDs**) in each TLB entry – uniquely identifies each process to provide address-space protection for that process
  - Otherwise need to flush at every context switch
- TLBs typically small (64 to 1,024 entries)
- On a TLB miss, value is loaded into the TLB for faster access next time
  - Replacement policies must be considered
  - Some entries can be **wired down** for permanent fast access

Memory protection

The OS restricts which memory a process may read, write, or execute.

Valid / Invalid

Is this page legal for the process?

Read / Write

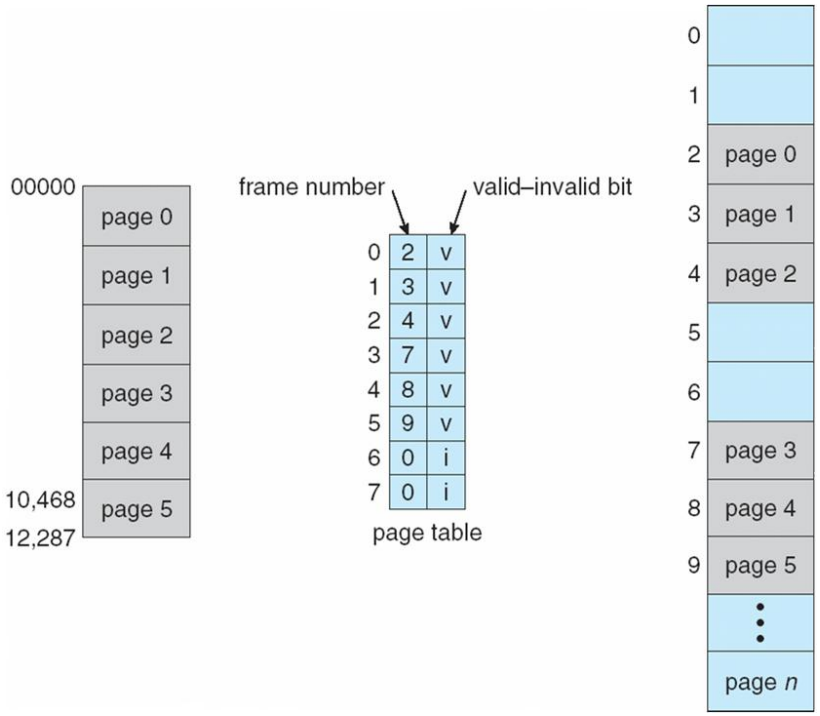
Can data be modified?

Execute

Can instructions run from this page?

Memory protection prevents one process from corrupting another process or the kernel.

Silberschatz, Galvin and Gagne ©2018, Operating System Concepts – 10th Edition



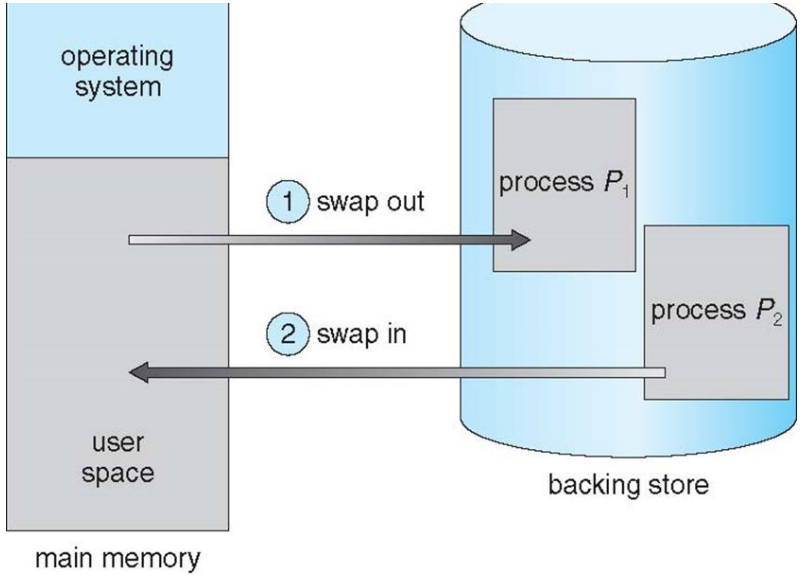
Memory Protection

- Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed
  - Can also add more bits to indicate page execute-only, and so on
- **Valid-invalid** bit attached to each entry in the page table:
  - "valid" indicates that the associated page is in the process' logical address space, and is thus a legal page
  - "invalid" indicates that the page is not in the process' logical address space
  - Or use **page-table length register** (**PTLR**)
- Any violations result in a trap to the kernel

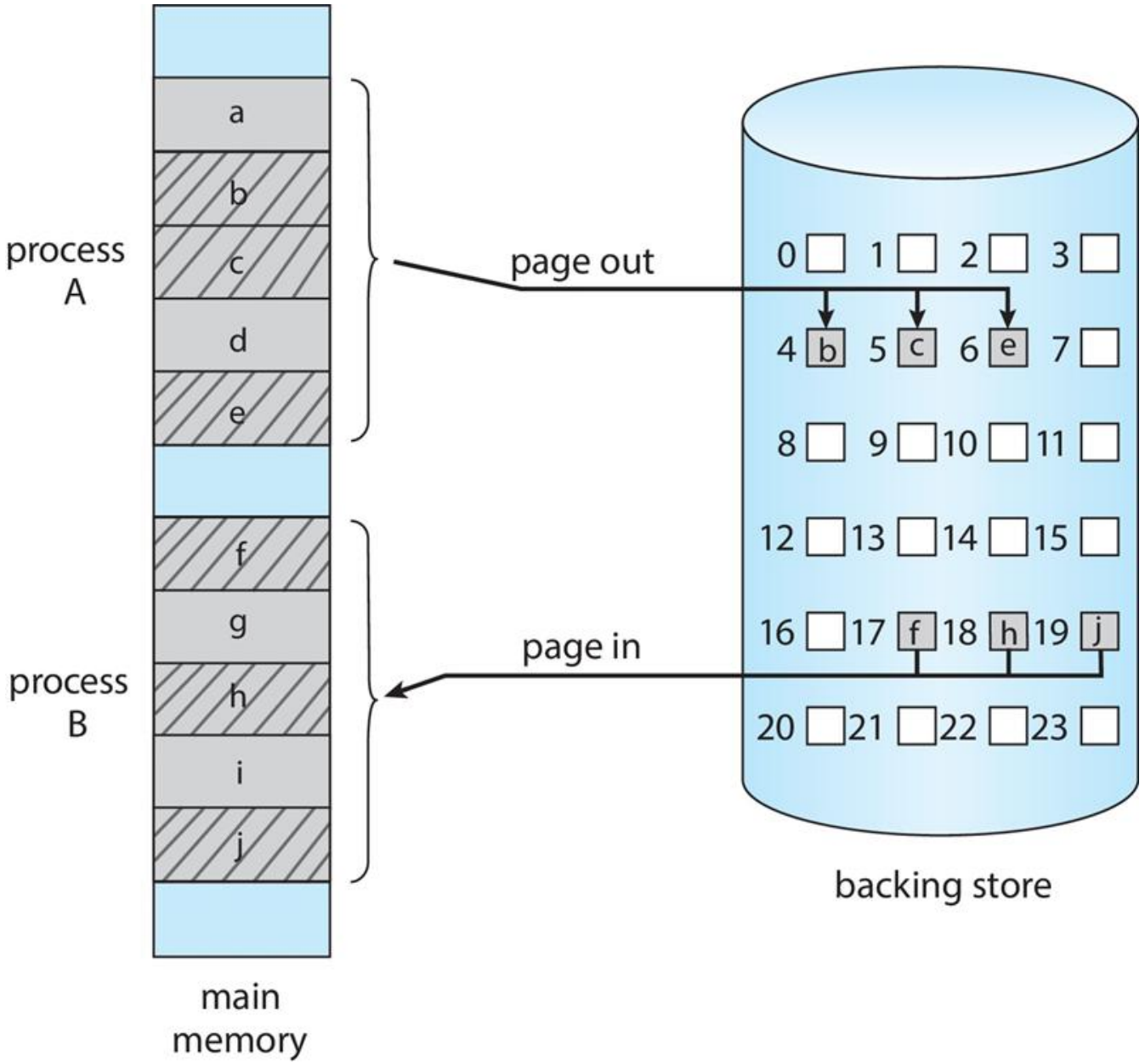
Silberschatz, Galvin and Gagne ©2018, Operating System Concepts – 10th Edition

Swapping

- A process can be **swapped** temporarily out of memory to a backing store, and then brought **back** into memory for continued execution
  - Total physical memory space of processes can exceed physical memory
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk



Swapping on Mobile Systems



06

File Management

How does the OS turn raw storage into useful files?



What is a file?

A file is an OS abstraction over stored bytes with attributes.

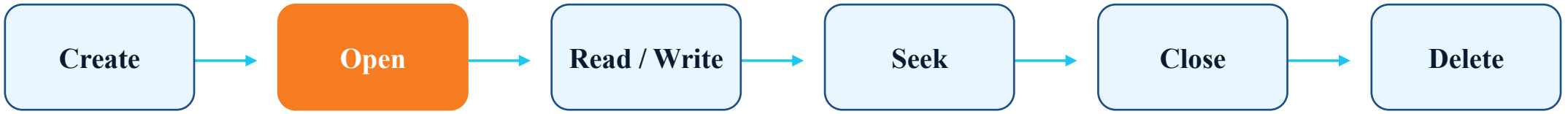


- **Name**
- Type
- Location
- Size
- Protection
- Time / date / owner

File attributes are kept in directory structures maintained on storage.

File operations and open files

The OS tracks file activity while processes are using files.



- Open-file table tracks active files
- File pointer tracks read/write location
- Access rights define allowed operations

Examples of file systems

Different operating-system families use different file-system designs.

Microsoft	Linux	macOS
FAT FAT32 NTFS	Ext2 Ext3	HFS HFS+

Examples from course material; not an exhaustive list of all current file systems.

Exit Ticket

1. Why do applications need an OS?
2. Interrupt vs DMA?
3. Program vs process?
4. Why CPU scheduling?
5. What problem does paging solve?

Next week: Virtual Machines and Cloud Computing

What if one physical computer could behave like many computers?

Source: Adapted from course materials and Silberschatz, Galvin & Gagne, Operating System Concepts, 10th ed. (2018).